

Team notebook

HSG-DiagonalBottle

August 3, 2026

Contents

1 Algorithms	1
1.1 Hash	1
1.2 Mo's algorithm on trees	1
1.3 Mo's algorithm	1
2 DP Optimizations	2
2.1 LineContainer	2
2.2 convex hull trick	2
2.3 divide and conquer	2
2.4 dp on trees	2
3 Data structures	3
3.1 Centroid	3
3.2 ChairmanTree	4
3.3 Fenwick 2D Offline	4
3.4 HLD	4
3.5 PerFen	5
3.6 PerTrie	6
3.7 STL order statistics tree	6
3.8 segment tree	6
3.9 sparse table	7
3.10 trie	7
4 Geometry	8
4.1 Notes	8
4.2 center 2 points + rarious	8
4.3 closest pair problem	8
4.4 convex hull	8
4.5 squares	9
4.6 triangles	9
5 Graphs	9
5.1 2ECC	9
5.2 BCTree	10
5.3 Dinic	10
5.4 Edmonds-Karp	11
5.5 JointBridge	11
5.6 MinCostFlow	11
5.7 SCC	12
5.8 dijkstra	13

5.9 directed mst	13
5.10 eulerian path	13
5.11 karp min mean cycle	14
5.12 magic max flow	14
5.13 maximum matching	14
5.14 minimum path cover in DAG	15
5.15 query with lca	15
5.16 two sat (with kosaraju)	15
6 Math	16
6.1 Lucas theorem	16
6.2 cumulative sum of divisors	16
6.3 fibonacci properties	16
6.4 sigma function	17
6.5 system of equations	17
7 Matrix	17
7.1 MaxMul	17
8 Misc	17
8.1 dates and bit	17
9 Number theory	18
9.1 Factorizer	18
9.2 crt	18
9.3 diophantine equations	18
9.4 discrete logarithm	19
9.5 ext euclidean	19
9.6 highest exponent factorial	19
9.7 miller rabin	19
9.8 mod integer	19
9.9 mod inv	20
9.10 mod mul	20
9.11 primes	20
9.12 totient sieve	20
9.13 totient	20
10 Strings	20
10.1 Incremental Aho Corasick	20
10.2 KMP	21
10.3 ZFunc	21
10.4 manacher	21

10.5 minimal string rotation	22
10.6 suffix array	22
10.7 suffix automaton	22
11 X - Extra Math	23
11.1 equations	23
11.2 Trigonometry and Triangles	23
11.3 Quadrilaterals	23
11.4 Spherical coordinates	23
11.5 Sums and (Geometric) series	23
11.6 Combinatorics	23
11.7 matrices	24
11.8 vectors	24
12 Z - Templates	25
12.1 stress	25

1 Algorithms

1.1 Hash

```
// Recommended to #define int ll here.
struct Hash {
    Hash() {}
    const int BASE = 53;
    vi pr = {1000000000 + 7, 998244353, 4518541637,
             7385466377};
    int mod;
    int n;
    vi hash, pw;
    Hash(int p, string s) {
        n = s.size();
        mod = pr[p];
        s = "@" + s;

        pw.resize(n + 1);
        hash.resize(n + 1);
        pw[0] = 1;
        for(int i = 1; i <= n; i++) pw[i] = (pw[i - 1] *
            BASE) % mod;
        for(int i = 1; i <= n; i++) hash[i] = ((hash[i - 1]
            * BASE) + (s[i] - 'a' + 1)) % mod;
    }
}
```

```

int get(int l, int r) {
    return (hash[r] - hash[l - 1] * pw[r - l + 1] + mod
            * mod) % mod;
}
};

```

1.2 Mo's algorithm on trees

```

/**
problems:
- https://codeforces.com/gym/101161 problem E
*/
void flat(vector<vector<edge>> &g, vector<int> &a,
          vector<int> &le, vector<int> &ri, vector<int> &cost,
          int node, int pi, int &ts, int w) {

    cost[node] = w;
    le[node] = ts;
    a[ts] = node;
    ts++;
    for (auto e : g[node]) {
        if (e.to == pi) continue;
        flat(g, a, le, ri, cost, e.to, node, ts, e.w);
    }
    ri[node] = ts;
    a[ts] = node;
    ts++;
}

/**
 * Case when the cost is in the edges.
 */
void compute_queries(vector<vector<edge>> &g) {
    // g is undirected
    int n = g.size();

    lca_tree.init(g, 0);

    vector<int> a(2 * n), le(n), ri(n), cost(n);
    // a: nodes in the flatten array
    // le: left id of the given node
    // ri: right id of the given node
    // cost: cost of the edge from the node to the parent

    int ts = 0; // timestamp
    flat(g, a, le, ri, cost, 0, -1, ts, 0);

    int q; cin >> q;
    vector<Query> queries(q);
    for (int i = 0; i < q; i++) {
        int u, v;
        cin >> u >> v;
        u--; v--;
        int lca = lca_tree.query(u, v);
        if (le[u] > le[v])
            swap(u, v);
        queries[i].id = i;
        queries[i].lca = lca;
    }
}

```

```

queries[i].u = u;
queries[i].v = v;
if (lca == u) {
    queries[i].a = le[u] + 1;
    queries[i].b = le[v];
} else {
    queries[i].a = ri[u];
    queries[i].b = le[v];
}
}
solve_mo(queries, a, le, cost); // this is the usual
algorithm
}

```

1.3 Mo's algorithm

```

const int MN = 5 * 100000 + 1;
const int SN = 708;

struct Query {
    int a, b, id;
    Query() {}
    Query(int x, int y, int i) : a(x), b(y), id(i) {}

    bool operator<(const Query &o) const {
        if (a / SN != o.a / SN) return a < o.a;
        return a / SN & 1 ? b < o.b : b > o.b;
    }
};

struct DS {
    DS() : {}

    void Insert(int x) {}

    void Erase(int x) {}

    long long Query() {}
};

Query s[MN];
int ans[MN];
DS active;

int main() {
    int n;
    cin >> n;
    vector<int> a(n);
    for (auto &i : a) cin >> i;

    int q;
    cin >> q;
    for (int i = 0; i < q; ++i) {
        int b, e;
        cin >> b >> e;
        b--;
        e--;
        s[i] = Query(b, e, i);
    }
}

```

```

sort(s, s + q);

int i = 0;
int j = -1;
for (int k = 0; k < (int)q; ++k) {
    int L = s[k].a;
    int R = s[k].b;

    while (j < R) active.Insert(a[++j]);
    while (j > R) active.Erase(a[j--]);
    while (i < L) active.Erase(a[i++]);
    while (i > L) active.Insert(a[--i]);

    ans[s[k].id] = active.Query();
}

for (int i = 0; i < q; ++i) {
    cout << ans[i] << endl;
}

return 0;
};

```

2 DP Optimizations

2.1 LineContainer

```

struct LineContainer
{
    struct line{
        ll a, b;
        mutable ll p;
        bool operator < (const line& o) const {
            if(o.a == INF && o.b == INF) return p < o.p;
            return a < o.a;
        }
    };
    multiset<line> lc;
    ll div(ll a, ll b){
        return a / b - ((a ^ b) < 0 && a % b);
    }
    bool phe(multiset<line>::iterator x,
             multiset<line>::iterator y){
        if(y == lc.end()){
            x->p = INF;
            return 0;
        }
        if(x->a == y->a){
            if(x->b >= y->b) x->p = INF;
            else x->p = -INF;
            return x->p >= y->p;
        }
        x->p = div(x->b - y->b, y->a - x->a);
        return x->p >= y->p;
    }
    void add(ll a, ll b){
        auto x = lc.insert({a, b, 0});
    }
}

```

```

    auto y = next(x);
    while(phe(x, y)) y = lc.erase(y);
    if((y = x) != lc.begin() && phe(--x, y))
        phe(x, y = lc.erase(y));
    while((y = x) != lc.begin() && (--x)->p >= y->p)
        phe(x, y = lc.erase(y));
}
ll qr(ll x){
    assert(!lc.empty());
    line l = *lc.lower_bound({INF, INF, x});
    return 1.a * x + 1.b;
}
};

```

2.2 convex hull trick

```

struct line {
    long long m, b;
    line (long long a, long long c) : m(a), b(c) {}
    long long eval(long long x) {
        return m * x + b;
    }
};

long double inter(line a, line b) {
    long double den = a.m - b.m;
    long double num = b.b - a.b;
    return num / den;
}

/**
 * min m_i * x_j + b_i, for all i.
 * x_j <= x_{j+1}
 * m_i >= m_{j+1}
 */
struct ordered_cht {
    vector<line> ch;
    int idx; // id of last "best" in query
    ordered_cht() {
        idx = 0;
    }

    void insert_line(long long m, long long b) {
        line cur(m, b);
        // new line's slope is less than all the previous
        while (ch.size() > 1 &&
            (inter(cur, ch[ch.size() - 2]) >= inter(cur,
                ch[ch.size() - 1]))) {
            // f(x) is better in interval [inter(ch.back(),
                cur), inf)
            ch.pop_back();
        }

        ch.push_back(cur);
    }

    long long eval(long long x) { // minimum
        // current x is greater than all the previous x,
        // if that is not the case we can make binary search.

```

```

        idx = min<int>(idx, ch.size() - 1);
        while (idx + 1 < (int)ch.size() && ch[idx + 1].eval(x)
            <= ch[idx].eval(x))
            idx++;

        return ch[idx].eval(x);
    }
};

```

2.3 divide and conquer

```

/**
 * recurrence:
 * dp[k][i] = min dp[k-1][j] + c[i][j - 1], for all j > i;
 *
 * "comp" computes dp[k][i] for all i in O(n log n) (k is fixed)
 */

void comp(int l, int r, int le, int re) {
    if (l > r) return;

    int mid = (l + r) >> 1;

    int best = max(mid + 1, le);
    dp[cur][mid] = dp[cur ^ 1][best] + cost(mid, best - 1);
    for (int i = best; i <= re; i++) {
        if (dp[cur][mid] > dp[cur ^ 1][i] + cost(mid, i - 1)) {
            best = i;
            dp[cur][mid] = dp[cur ^ 1][i] + cost(mid, i - 1);
        }
    }

    comp(l, mid - 1, le, best);
    comp(mid + 1, r, best, re);
}

```

2.4 dp on trees

```

/**
 * This trick is very useful when doing DP on trees,
 * basically, you can save
 * the answer for each node as if it was the root of the
 * tree. Partial results
 * are also stored in order to query subtrees (taking the
 * root and exclude some
 * child).
 */

struct edge {
    int to, p_id;
    edge (int a, int b) : to(a), p_id(b) {}
};

```

```

struct state {
    bool seen;
    long long missing;
    long long total;
    vector<long long> partial;

    state() { clear(); }

    void clear() {
        seen = false;
        missing = 0;
        total = 0;
        partial.clear();
    }
};

void add_edge(int u, int v) {
    int id_u_v = g[u].size();
    int id_v_u = g[v].size();
    g[u].emplace_back(v, id_v_u); // id of the parent in the
    // child's list (g[v][id] -> u)
    g[v].emplace_back(u, id_u_v); // id of the parent in the
    // child's list (g[u][id] -> v)
}

int go(int node, int id_parent) {

    state &s = dp[node];

    if (!s.seen) {
        int ans = 1;
        s.partial.assign(g[node].size(), 0); // create the list
        // of partial results.
        for (int i = 0; i < (int)g[node].size(); i++) {
            int to = g[node][i].to;
            int pid = g[node][i].p_id;
            if (i != id_parent) {
                int tmp = go(to, pid);
                ans += tmp;
                s.partial[i] = tmp;
            }
        }

        s.missing = id_parent;
        s.total = ans;
        s.seen = true;

        return ans;
    } else {

        if (s.missing == id_parent) { // the same id_parent
            // than before, so we can not complete the results
            // yet
            return s.total;
        }

        if (s.missing != -1) { // only one missing and is
            // different of 'id_parent'
            int tmp = go(g[node][s.missing].to,
                g[node][s.missing].p_id);

```

```

    s.partial[s.missing] = tmp;
    s.total += tmp;
    s.missing = -1;
}

int extra = (id_parent == -1) ? 0 :
    s.partial[id_parent];
return s.total - extra;
}
}

```

3 Data structures

3.1 Centroid

```

// Born_To_Laugh - Hughie Do
#include <bits/stdc++.h>
#define alle(AC) AC.begin(), AC.end()
#define fi first
#define se second
using namespace std;
typedef long long ll;
[[maybe_unused]] const ll MOD = 998244353, INF = 1e9 + 7;
const int maxn = 1e5 + 10;
int n, q;
vector<int> adj[maxn];
int dep[maxn], del[maxn], sz[maxn], binlift[maxn][20];
int paric[maxn];
vector<int> mintor(maxn, INF);
void dfs(int a, int par){
    for(auto &elm: adj[a]){
        if(elm == par)continue;
        binlift[elm][0] = a;
        for(int i=1; i<20; ++i){
            binlift[elm][i] =
                binlift[binlift[elm][i-1]][i-1];
        }
        dep[elm] = dep[a] + 1;
        dfs(elm, a);
    }
}
int lca(int a, int b){
    if(dep[a] < dep[b])swap(a, b);
    int k = dep[a] - dep[b];
    for(int i=0; i<20; ++i){
        if(k & (1 << i)){
            a = binlift[a][i];
        }
    }
    if(a == b)return a;
    for(int i=19; i>=0; --i){
        if(binlift[a][i] != binlift[b][i]){
            a = binlift[a][i];
            b = binlift[b][i];
        }
    }
    return binlift[a][0];
}

```

```

inline int getdist(int a, int b){
    return dep[a] + dep[b] - dep[lca(a, b)] * 2;
}
void predfs(int a, int par){
    sz[a] = 1;
    for(auto &elm: adj[a]){
        if(del[elm] || elm == par)continue;
        predfs(elm, a);
        sz[a] += sz[elm];
    }
}
int findcent(int a, int par, int root){
    for(auto &elm: adj[a]){
        if(elm == par || del[elm])continue;
        if(sz[elm] > sz[root]/2)return findcent(elm, a,
            root);
    }
    return a;
}
void decom(int a, int par){
    predfs(a, -1);
    int cent = findcent(a, -1, a);
    del[cent] = 1;
    paric[cent] = par;
    for(auto &elm: adj[cent]){
        if(del[elm])continue;
        decom(elm, cent);
    }
}
void ope1(int a){
    int b = a;
    while(b != -1){
        mintor[b] = min(mintor[b], getdist(a, b));
        b = paric[b];
    }
}
int ope2(int a){
    int b = a;
    int ans = INF;
    while(b != -1){
        ans = min(ans, getdist(a, b) + mintor[b]);
        b = paric[b];
    }
    return ans;
}
void solve(){
    cin >> n >> q;
    for(int i=1; i<n; ++i){
        int a, b;cin >> a >> b;
        adj[a].push_back(b);
        adj[b].push_back(a);
    }
    dfs(1, -1);
    decom(1, -1);
    ope1(1);
    while(q--){
        int sth;cin >> sth;
        if(sth & 1){
            int x;cin >> x;
            ope1(x);
        }
        else{

```

```

            int x;cin >> x;
            cout << ope2(x) << '\n';
        }
    }
}
signed main(){
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    solve();
}

```

3.2 ChairmanTree

```

struct PersistentSegTree {
    struct Node {
        int val = 0;
        int l = -1, r = -1;

        Node() {}
        Node(int v) : val(v) {}
        Node(int v, int L, int R) : val(v), l(L), r(R) {}

        Node operator+(const Node &a, const Node &b) {
            return Node(a.val + b.val, a.l, b.r);
        }
    };

    vector<Node> st;
    int n, id;

    void init(int _n){
        n = _n;
        st.resize(20 * _n + 5);
    }

    int add_copy(int root) {
        st.push_back(st[root]);
        return id++;
    }

    int build(int l, int r) {
        int my = id++;
        st.emplace_back();
        if (l == r) return my;

        int mid = (l + r) >> 1;
        int left = build(l, mid);
        int right = build(mid + 1, r);
        st[my] = st[left] + st[right];
        st[my].l = left;
        st[my].r = right;
        return my;
    }

    int update(int cur, int l, int r, int k, int x) {
        if (k < l || k > r) return cur;
        if (l == r) {
            st.emplace_back(x);
            return id++;
        }
    }
}

```

```

}

int mid = (l + r) >> 1;
int newLeft = update(st[cur].l, l, mid, k, x);
int newRight = updateRec(st[cur].r, mid + 1, r, k, x);
st.emplace_back(st[newLeft] + st[newRight]);
st.back().l = newLeft;
st.back().r = newRight;
return id++;
}

int queryRec(int cur, int l, int r, int L, int R) {
    if (R < l || r < L) return 0;
    if (L <= l && r <= R) return st[cur].val;
    int mid = (l + r) >> 1;
    return queryRec(st[cur].l, l, mid, L, R) +
           queryRec(st[cur].r, mid + 1, r, L, R);
}
};

```

3.3 Fenwick 2D Offline

```

struct OfflineFenwickTree2D {
    int n;
    vector<vi> vals;
    vector<vi> bit;

    int id(vector<int> &v, int x) {
        return lower_bound(v.begin(), v.end(), x) -
               v.begin() + 1;
    }

    OfflineFenwickTree2D(int n): n(n), vals(n + 1), bit(n + 1) {}

    void fakeUse(int x, int y) {
        for(; x <= n; x += x & (-x)) {
            vals[x].push_back(y);
        }
    }

    void init() {
        for(int i = 1; i <= n; i++) {
            sort(vals[i].begin(), vals[i].end());
            vals[i].erase(unique(vals[i].begin(),
                                vals[i].end(), vals[i].end()),
                           vals[i].end());
            bit[i].resize(vals[i].size() + 5);
        }
    }

    void upd(int x, int y, int w) {
        for(; x <= n; x += x & (-x)) {
            for(int i = id(vals[x], y); i < bit[x].size();
                i += i & (-i)) {
                bit[x][i] = max(bit[x][i], w);
            }
        }
    }
};

```

```

int query(int x, int y) {
    int res = 0;
    for(; x > 0; x -= x & (-x)) {
        for(int i = id(vals[x], y); i > 0; i -= i & (-i)) {
            res = max(res, bit[x][i]);
        }
    }
    return res;
}
};

```

3.4 HLD

```

// Born_To_Laugh - Hughie Do
#include <bits/stdc++.h>
#define alle(AC) AC.begin(), AC.end()
using namespace std;
typedef long long ll;
[[maybe_unused]] const ll MOD = 998244353, INF = 1e9 + 7;
const int maxn = 5e4 + 10;
class segtree
{
private:
    int n;
    vector<int> tree;
    vector<int> lazy;
    void operate(int id, int l, int r){
        if(!lazy[id])return;
        tree[id] += lazy[id];
        if(l != r){
            lazy[id << 1] += lazy[id];
            lazy[id << 1|1] += lazy[id];
        }
        lazy[id] = 0;
    }
    void upd(int id, int l, int r, int lo, int hi, int val){
        operate(id, l, r);
        if(l > hi || r < lo)return;
        else if(l >= lo && r <= hi){
            lazy[id] += val;
            operate(id, l, r);
            return;
        }
        int mid = (l + r) >> 1;
        upd(id << 1, l, mid, lo, hi, val);
        upd(id << 1|1, mid+1, r, lo, hi, val);
        tree[id] = max(tree[id << 1], tree[id << 1|1]);
    }
public:
    segtree(){
        n = maxn;
        tree.assign(n << 2, 0);
        lazy.assign(n << 2, 0);
    }
    void upd(int lo, int hi){
        upd(1, 1, n, lo, hi, 1);
    }
};

```

```

int get(){
    return tree[1];
}

};

segtree st;
int n, q;
vector<int> adj[maxn];
int tin[maxn], chain[maxn], head[maxn], parr[maxn],
    big[maxn], dep[maxn], sz[maxn];
int id = 0;
int cur = 1;
void predfs(int a, int par){
    sz[a] = 1;
    for(auto &elm: adj[a]){
        if(elm == par)continue;
        dep[elm] = dep[a] + 1;
        predfs(elm, a);
        sz[a] += sz[elm];
        if(sz[elm] > sz[big[a]])big[a] = elm;
    }
}

void hld(int a, int par, int root){
    tin[a] = ++id;
    chain[a] = cur;
    parr[a] = par;
    head[a] = root;
    if(big[a])hld(big[a], a, root);
    for(auto &elm: adj[a]){
        if(elm == par || elm == big[a])continue;
        cur++;
        hld(elm, a, elm);
    }
}

int lca(int a, int b){
    while(chain[a] != chain[b]){
        if(chain[a] < chain[b])swap(a, b);
        a = parr[head[a]];
    }
    if(dep[a] > dep[b])swap(a, b);
    return a;
}

void ope(int a, int b){
    int c = lca(a, b);
    // cout << a << ' ' << b << ' ' << c << '\n';
    // return;
    // int c = 0;
    while(chain[a] != chain[c]){
        st.upd(tin[head[a]], tin[a]);
        a = parr[head[a]];
    }
    while(chain[b] != chain[c]){
        st.upd(tin[head[b]], tin[b]);
        b = parr[head[b]];
    }
    if(dep[a] > dep[b])swap(a, b);
    st.upd(tin[a], tin[b]);
}

void solve(){
    cin >> n >> q;
    for(int i=1; i<n; ++i){
        int a, b;cin >> a >> b;
        adj[a].push_back(b);
    }
}

```

```

    adj[b].push_back(a);
}
predfs(1, -1);
hld(1, -1, 1);
for(int i=1; i<=q; ++i){
    int a, b; cin >> a >> b;
    ope(a, b);
}
cout << st.get();
}
signed main(){
    freopen("maxflow.in", "r", stdin);
    freopen("maxflow.out", "w", stdout);
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    solve();
}

```

3.5 PerFen

```

// Born_To_Laugh - Hughie Do
#include <bits/stdc++.h>
#define alle(AC) AC.begin(), AC.end()
#define fi first
#define se second
using namespace std;
typedef long long ll;
[[maybe_unused]] const int MOD = 998244353, INF = 1e9 + 7;
const int maxn = 2e5 + 10;

struct PerFen
{
    int n, time;
    vector<vector<pair<int, int>>> bit;
    void init(int len){
        n = len;
        time = 0;
        bit.assign(len + 1, {{0, 0}});
    }
    int upd(int pos, int val){
        time++;
        for(; pos<=n; pos += pos & -pos){
            bit[pos].push_back({time, bit[pos].back().se + val});
        }
        return time;
    }
    int qr(int pos, int tim){
        int ans = 0;
        for(; pos>0; pos -= pos & -pos){
            int id = upper_bound(alle(bit[pos]),
                                make_pair(tim, INF)) - bit[pos].begin() - 1;
            ans += bit[pos][id].se;
        }
        return ans;
    }
    int qr(int l, int r, int pos){
        if(l > r) return 0;

```

```

        return qr(r, pos) - qr(l - 1, pos);
    }
};

int n, k;
int a[maxn], b[maxn], t[maxn];
int prefsum[maxn * 2];
int ver[maxn];
PerFen ft;
vector<int> temp;

void solve(){
    cin >> n >> k;
    for(int i=1; i<=n; ++i){
        cin >> a[i] >> b[i];
        temp.push_back(a[i]);
        temp.push_back(b[i]);
    }
    for(int i=1; i<=k; ++i){
        cin >> t[i];
        temp.push_back(t[i]);
    }
    sort(alle(temp));
    temp.erase(unique(alle(temp)), temp.end());
    ft.init(temp.size() + 5);

    for(int i=1; i<=n; ++i){
        a[i] = lower_bound(alle(temp), a[i]) - temp.begin() + 1;
        b[i] = lower_bound(alle(temp), b[i]) - temp.begin() + 1;
    }
    for(int i=1; i<=k; ++i){
        t[i] = lower_bound(alle(temp), t[i]) - temp.begin() + 1;
    }
    for(int i=k; i>=1; --i) ver[i] = ft.upd(t[i], 1);

    auto check = [&](int x, int a, int b) -> bool {
        return ft.qr(a, b - 1, ver[x]) == 0;
    };

    ll ans = 0;
    for(int i=1; i<=n; ++i){
        if(ft.qr(min(a[i], b[i]), max(a[i], b[i]) - 1, ver[i]) == 0){
            int num = ft.qr(max(a[i], b[i]), ft.n, ver[i]);
            if(num & 1){
                ans += temp[b[i] - 1];
            }
            else ans += temp[a[i] - 1];
            continue;
        }

        int lo = 1, hi = k;
        int res = hi;
        while(hi >= lo){
            int mid = (lo + hi) >> 1;
            if(check(mid, min(a[i], b[i]), max(a[i], b[i]))){
                res = mid;
                hi = mid - 1;

```

```

            }
            else lo = mid + 1;
        }

        int num = ft.qr(max(a[i], b[i]), ft.n, ver[res]);
        if(num & 1){
            ans += temp[min(a[i], b[i]) - 1];
        }
        else{
            ans += temp[max(a[i], b[i]) - 1];
        }
    }
    cout << ans << '\n';
}

signed main(){
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    solve();
}

```

3.6 PerTrie

```

struct PerTrie{
    struct node{
        int cnt = 0;
        array<int, 2> child;
        node(){
            child.fill(-1);
        }
    };
    int id = 0;
    vector<node> tree;
    PerTrie(){
        tree.emplace_back();
    }
    int new_node(){
        ++id;
        if(id >= tree.size()) tree.emplace_back();
        return id;
    }
    int add(int a, int prev){
        int cc = new_node();
        int cur = cc;
        tree[cur] = tree[prev];
        tree[cur].cnt++;
        for(int i=30; i>=0; --i){
            int val = (a >> i) & 1;
            int nxt = new_node();

            if(prev != -1 && tree[prev].child[val] != -1){
                tree[nxt] = tree[tree[prev].child[val]];
                prev = tree[prev].child[val];
            }
            else prev = -1;

            tree[cur].child[val] = nxt;
            cur = nxt;
            tree[cur].cnt++;
        }
    }
}

```

```

    return cc;
}

int walk(int prev, int cur, int a){
    int ans = 0;
    for(int i=30; i>=0; --i){
        int val = 1 - ((a >> i) & 1);
        int num1 = (cur != -1 && tree[cur].child[val]
            != -1) ? tree[tree[cur].child[val]].cnt :
            0;
        int num2 = (prev != -1 && tree[prev].child[val]
            != -1) ? tree[tree[prev].child[val]].cnt :
            0;
        if(num1 - num2 > 0){
            ans |= (val << i);
            cur = (cur != -1) ? tree[cur].child[val] :
                -1;
            prev = (prev != -1) ? tree[prev].child[val]
                : -1;
        }
        else{
            ans |= ((1 - val) << i);
            cur = (cur != -1) ? tree[cur].child[1 - val]
                : -1;
            prev = (prev != -1) ? tree[prev].child[1 -
                val] : -1;
        }
    }
    return ans;
}
};

```

3.7 STL order statistics tree

```

#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
#include <bits/stdc++.h>

```

```

using namespace __gnu_pbds;
using namespace std;

```

```

typedef
tree<
    pair<int,int>,
    null_type,
    less<pair<int,int>>,
    rb_tree_tag,
    tree_order_statistics_node_update>
ordered_set;

```

```

main()
{
    ios::sync_with_stdio(0);
    cin.tie(0);
    int n;
    int sz=0;
    cin>>n;
    vector<int> ans(n,0);

    ordered_set t;

```

```

    int x,y;
    for(int i=0;i<n;i++){
        cin>>x>>y;
        ans[t.order_of_key({x,++sz})]++;
        t.insert({x,sz});
    }

    for(int i=0;i<n;i++){
        cout<<ans[i]<<'\\n';
    }

    /**
    Input
    5
    1 1
    5 1
    7 1
    3 3
    5 5

    Output
    1
    2
    1
    1
    0
    ***/

```

3.8 segment tree

```

/**
 * Taken from: http://codeforces.com/blog/entry/18051
 */

const int MN = 1e5; // limit for array size

struct seg_tree {
    int n; // array size
    int t[2 * MN];

    seg_tree(int _n) : n(_n) {}

    void clear() {
        memset(t, 0, sizeof t);
    }

    void build() { // build the tree
        for (int i = n - 1; i > 0; --i) t[i] = t[i<<1] +
            t[i<<1|1];
    }

    // Single modification, range query.
    void modify(int p, int value) { // set value at position
        p
        for (t[p += n] = value; p > 1; p >>= 1) t[p>>1] = t[p]
            + t[p^1];
    }
}

```

```

int query(int l, int r) { // sum on interval [l, r]
    int res = 0;
    for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
        if (l&1) res += t[l++];
        if (r&1) res += t[--r];
    }
    return res;
}

// Range modification, single query.

void modify(int l, int r, int value) {
    for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
        if (l&1) t[l++] += value;
        if (r&1) t[--r] += value;
    }
}

int query(int p) {
    int res = 0;
    for (p += n; p > 0; p >>= 1) res += t[p];
}

/**
 * If at some point after modifications we need to inspect
 * all the
 * elements in the array, we can push all the
 * modifications to the
 * leaves using the following code. After that we can just
 * traverse
 * elements starting with index n. This way we reduce the
 * complexity
 * from O(n log(n)) to O(n) similarly to using build
 * instead of n modifications.
 */

void push() {
    for (int i = 1; i < n; ++i) {
        t[i<<1] += t[i];
        t[i<<1|1] += t[i];
        t[i] = 0;
    }
}

// Non commutative combiner functions.

void modify(int p, const S& value) {
    for (t[p += n] = value; p >>= 1; ) t[p] =
        combine(t[p<<1], t[p<<1|1]);
}

S query(int l, int r) {
    S resl, resr;
    for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
        if (l&1) resl = combine(resl, t[l++]);
        if (r&1) resr = combine(t[--r], resr);
    }
    return combine(resl, resr);
}

```

```

/**
 * segment tree for intervals
 */

const int MN = 100000 + 100;
struct seg_tree {
    int val[MN * 4 + 4];
    int pending[MN * 4 + 4];

    seg_tree() {
        memset(val, -1, sizeof val);
        memset(pending, -1, sizeof pending);
    }

    void propagate(int node, int b, int e) {
        if (pending[node] != -1) {
            val[node] = pending[node];
            if (b < e) {
                pending[node << 1] = pending[node];
                pending[node << 1 | 1] = pending[node];
            }
            pending[node] = -1;
        }
    }

    void set(int node, int b, int e, int from, int to, int v) {
        if (b > to || e < from) return;

        if (b >= from && e <= to) {
            pending[node] = v;
            propagate(node, b, e);
            return;
        }

        int mid = (b + e) >> 1;

        set(node << 1, b, mid, from, to, v);
        set(node << 1 | 1, mid + 1, e, from, to, v);
    }

    int query(int node, int b, int e, int pos) {
        propagate(node, b, e);

        if (b == e && b == pos) {
            return val[node];
        }

        int mid = (b + e) >> 1;
        if (pos <= mid)
            return query(node << 1, b, mid, pos);
        return query(node << 1 | 1, mid + 1, e, pos);
    }

    void set(int from, int to, int v) {
        return set(1, 0, MN - 1, from, to, v);
    }
}

```

```

int query(int pos) {
    return query(1, 0, MN - 1, pos);
}
};

```

3.9 sparse table

```

// RMQ.
const int MN = 100000 + 10; // Max number of elements
const int ML = 18; // ceil(log2(MN));

struct st {
    int data[MN];
    int M[MN][ML];
    int n;

    void init(const vector<int> &d) {
        n = d.size();
        for (int i = 0; i < n; ++i)
            data[i] = d[i];

        build();
    }

    void build() {
        for (int i = 0; i < n; ++i)
            M[i][0] = data[i];
        for (int j = 1, p = 2, q = 1; p <= n; ++j, p <= 1, q
            <= 1)
            for (int i = 0; i + p - 1 < n; ++i)
                M[i][j] = max(M[i][j - 1], M[i + q][j - 1]);
    }

    int query(int b, int e) {
        int k = log2(e - b + 1);
        return max(M[b][k], M[e + 1 - (1 << k)][k]);
    }
};

```

3.10 trie

```

const int MN = 26; // size of alphabet
const int MS = 100010; // Number of states.

struct trie {
    struct node {
        int c;
        int a[MN];
    };

    node tree[MS];
    int nodes;

    void clear() {
        tree[nodes].c = 0;
        memset(tree[nodes].a, -1, sizeof tree[nodes].a);
        nodes++;
    }
}

```

```

}

void init() {
    nodes = 0;
    clear();
}

int add(const string &s, bool query = 0) {
    int cur_node = 0;
    for (int i = 0; i < s.size(); ++i) {
        int id = gid(s[i]);
        if (tree[cur_node].a[id] == -1) {
            if (query) return 0;
            tree[cur_node].a[id] = nodes;
            clear();
        }
        cur_node = tree[cur_node].a[id];
    }
    if (!query) tree[cur_node].c++;
    return tree[cur_node].c;
}
};

```

4 Geometry

4.1 Notes

Area of a polygon is:

$$\frac{1}{2} \sum_{i=1}^{n-1} |p_i \times p_{i+1}| = \frac{1}{2} \sum_{i=1}^{n-1} |x_i y_{i+1} - x_{i+1} y_i|$$

p_i is adjacent to p_{i+1} , and $p_1 = p_n$.

If no absolute, if area is smaller than 0 then points are listed CCW, otherwise CW, if equals 0 then 3 points are in a line.

Point location to a segment. Point is p , segment from a to b ($p - a$) $\times$ ($p - b$) $< 0, = 0, > 0$: Left, Touch, Right

4.2 center 2 points + radius

```

vector<point> find_center(point a, point b, long double r)
{
    point d = (a - b) * 0.5;
    if (d.dot(d) > r * r) {
        return vector<point> ();
    }
    point e = b + d;
    long double fac = sqrt(r * r - d.dot(d));
    vector<point> ans;
    point x = point(-d.y, d.x);
    long double l = sqrt(x.dot(x));
    x = x * (fac / l);
    ans.push_back(e + x);
    x = point(d.y, -d.x);
    ans.push_back(e + x);
}

```



```

x = x * (fac / l);
ans.push_back(e + x);
return ans;
}

```

4.3 closest pair problem

```

struct point {
    double x, y;
    int id;
    point() {}
    point (double a, double b) : x(a), y(b) {}
};

```

```

double dist(const point &o, const point &p) {
    double a = p.x - o.x, b = p.y - o.y;
    return sqrt(a * a + b * b);
}

```

```

double cp(vector<point> &p, vector<point> &x,
    vector<point> &y) {
    if (p.size() < 4) {
        double best = 1e100;
        for (int i = 0; i < p.size(); ++i)
            for (int j = i + 1; j < p.size(); ++j)
                best = min(best, dist(p[i], p[j]));
        return best;
    }
}

```

```

int ls = (p.size() + 1) >> 1;
double l = (p[ls - 1].x + p[ls].x) * 0.5;
vector<point> xl(ls), xr(p.size() - ls);
unordered_set<int> left;
for (int i = 0; i < ls; ++i) {
    xl[i] = x[i];
    left.insert(x[i].id);
}
for (int i = ls; i < p.size(); ++i) {
    xr[i - ls] = x[i];
}

```

```

vector<point> yl, yr;
vector<point> pl, pr;
yl.reserve(ls); yr.reserve(p.size() - ls);
pl.reserve(ls); pr.reserve(p.size() - ls);
for (int i = 0; i < p.size(); ++i) {
    if (left.count(y[i].id))
        yl.push_back(y[i]);
    else
        yr.push_back(y[i]);

    if (left.count(p[i].id))
        pl.push_back(p[i]);
    else
        pr.push_back(p[i]);
}

```

```

double dl = cp(pl, xl, yl);
double dr = cp(pr, xr, yr);

```

```

double d = min(dl, dr);
vector<point> yp; yp.reserve(p.size());
for (int i = 0; i < p.size(); ++i) {
    if (fabs(y[i].x - l) < d)
        yp.push_back(y[i]);
}
for (int i = 0; i < yp.size(); ++i) {
    for (int j = i + 1; j < yp.size() && j < i + 7; ++j) {
        d = min(d, dist(yp[i], yp[j]));
    }
}
return d;
}

```

```

double closest_pair(vector<point> &p) {
    vector<point> x(p.begin(), p.end());
    sort(x.begin(), x.end(), [](const point &a, const point
        &b) {
            return a.x < b.x;
        });
    vector<point> y(p.begin(), p.end());
    sort(y.begin(), y.end(), [](const point &a, const point
        &b) {
            return a.y < b.y;
        });
    return cp(p, x, y);
}

```

4.4 convex hull

```

struct Point {
    int x, y;
    Point(int ix = 0, int iy = 0) {
        x = ix;
        y = iy;
    }
};

```

```

bool cmp(Point a, Point b) {
    if(a.x == b.x) return a.y < b.y;
    return a.x < b.x;
}

```

```

Point vec(Point p1, Point p2) {
    Point res;
    res.x = p2.x - p1.x;
    res.y = p2.y - p1.y;
    return res;
}

```

```

int cross(Point a, Point b) {
    return a.x * b.y - a.y * b.x;
}

```

```

int point_location(vector<Point> p) {
    int res = cross(vec(p[0], p[1]), vec(p[0], p[2]));
    if(res == 0) return 0;
    else if(res > 0) return -1; // left
    return 1; // right
}

```

```

}

void solve()
{
    int n; cin >> n;
    vector<Point> a(n);
    for(auto &it : a) cin >> it.x >> it.y;
    sort(a.begin(), a.end(), cmp);

    vector<Point> v1, v2;
    for(int i = 0; i < n; i++) {
        while(v1.size() >= 2 &&
            point_location({v1[(int)v1.size() - 2],
                v1[(int)v1.size() - 1], a[i]}) == -1)
            v1.pop_back();
        v1.push_back(a[i]);
    }
    for(int i = n - 1; i >= 0; i--) {
        while(v2.size() >= 2 &&
            point_location({v2[(int)v2.size() - 2],
                v2[(int)v2.size() - 1], a[i]}) == -1)
            v2.pop_back();
        v2.push_back(a[i]);
    }

    v1.pop_back();
    v2.pop_back();

    cout << v1.size() + v2.size() << '\n';
    for(auto v: {v1, v2}) {
        for(Point p : v) cout << p.x << ' ' << p.y << '\n';
    }
}

```

4.5 squares

```

typedef long double ld;

const ld eps = 1e-12;
int cmp(ld x, ld y = 0, ld tol = eps) {
    return (x <= y + tol) ? (x + tol < y) ? -1 : 0 : 1;
}

```

```

struct point{
    ld x, y;
    point(ld a, ld b) : x(a), y(b) {}
    point() {}
};

```

```

struct square{
    ld x1, x2, y1, y2,
    a, b, c;
    point edges[4];
    square(ld _a, ld _b, ld _c) {
        a = _a, b = _b, c = _c;
        x1 = a - c * 0.5;
        x2 = a + c * 0.5;
        y1 = b - c * 0.5;
    }
}

```

```

    y2 = b + c * 0.5;
    edges[0] = point(x1, y1);
    edges[1] = point(x2, y1);
    edges[2] = point(x2, y2);
    edges[3] = point(x1, y2);
}
};

ld min_dist(point &a, point &b) {
    ld x = a.x - b.x,
        y = a.y - b.y;
    return sqrt(x * x + y * y);
}

bool point_in_box(square s1, point p) {
    if (cmp(s1.x1, p.x) != 1 && cmp(s1.x2, p.x) != -1 &&
        cmp(s1.y1, p.y) != 1 && cmp(s1.y2, p.y) != -1)
        return true;
    return false;
}

bool inside(square &s1, square &s2) {
    for (int i = 0; i < 4; ++i)
        if (point_in_box(s2, s1.edges[i]))
            return true;

    return false;
}

bool inside_vert(square &s1, square &s2) {
    if ((cmp(s1.y1, s2.y1) != -1 && cmp(s1.y1, s2.y2) != 1)
        ||
        (cmp(s1.y2, s2.y1) != -1 && cmp(s1.y2, s2.y2) != 1))
        return true;
    return false;
}

bool inside_hori(square &s1, square &s2) {
    if ((cmp(s1.x1, s2.x1) != -1 && cmp(s1.x1, s2.x2) != 1)
        ||
        (cmp(s1.x2, s2.x1) != -1 && cmp(s1.x2, s2.x2) != 1))
        return true;
    return false;
}

ld min_dist(square &s1, square &s2) {
    if (inside(s1, s2) || inside(s2, s1))
        return 0;

    ld ans = 1e100;
    for (int i = 0; i < 4; ++i)
        for (int j = 0; j < 4; ++j)
            ans = min(ans, min_dist(s1.edges[i], s2.edges[j]));

    if (inside_hori(s1, s2) || inside_hori(s2, s1)) {
        if (cmp(s1.y1, s2.y2) != -1)
            ans = min(ans, s1.y1 - s2.y2);
        else
            if (cmp(s2.y1, s1.y2) != -1)
                ans = min(ans, s2.y1 - s1.y2);
    }
}

```

```

if (inside_vert(s1, s2) || inside_vert(s2, s1)) {
    if (cmp(s1.x1, s2.x2) != -1)
        ans = min(ans, s1.x1 - s2.x2);
    else
        if (cmp(s2.x1, s1.x2) != -1)
            ans = min(ans, s2.x1 - s1.x2);
}

return ans;
}

```

4.6 triangles

Let a, b, c be length of the three sides of a triangle.

$$p = (a + b + c) * 0.5$$

The inradius is defined by:

$$iR = \sqrt{\frac{(p-a)(p-b)(p-c)}{p}}$$

The radius of its circumcircle is given by the formula:

$$cR = \frac{abc}{\sqrt{(a+b+c)(a+b-c)(a+c-b)(b+c-a)}}$$

5 Graphs

5.1 2ECC

```

#include <bits/stdc++.h>
using namespace std;

const int MAX_N = 3e5;

int timer; // Time of entry in node
int scc; // Number of strongly connected components
int id[MAX_N];
int low[MAX_N]; // Lowest ID in node's subtree in DFS tree
vector<int> neighbors[MAX_N];
vector<int> two_edge_components[MAX_N];
stack<int> st; // Keeps track of the path in our DFS

void dfs(int node, int parent = -1) {
    id[node] = low[node] = ++timer;
    st.push(node);
    bool multiple_edges = false;

    for (int child : neighbors[node]) {
        if (child == parent && !multiple_edges) {
            multiple_edges = true;
            continue;
        }
    }
}

```

```

if (!id[child]) {
    dfs(child, node);
    low[node] = min(low[node], low[child]);
} else {
    low[node] = min(low[node], id[child]);
}

if (low[node] == id[node]) {
    while (st.top() != node) {
        two_edge_components[scc].push_back(st.top());
        st.pop();
    }
    two_edge_components[scc++].push_back(st.top());
    st.pop();
}

int main() {
    int n, m;
    cin >> n >> m;
    for (int i = 0; i < m; i++) {
        int x, y;
        cin >> x >> y;
        neighbors[x].push_back(y);
        neighbors[y].push_back(x);
    }

    for (int node = 0; node < n; node++) {
        if (id[node] == 0) { dfs(node); }
    }

    cout << scc << '\n';
    for (int i = 0; i < scc; i++) {
        cout << two_edge_components[i].size() << ' ';
        for (int node : two_edge_components[i]) {
            cout << node << ' ';
        }
        cout << '\n';
    }
}

```

5.2 BCTree

```

const int maxn = 2e5 + 10;
int tin[maxn], low[maxn], sz[maxn];
vector<int> adj[maxn], bccadj[maxn], st;
int id = 0, cc = 0;
int ccsz = 0;
int ans = 0;
int n, m;
void dfs(int a, int par) {
    tin[a] = low[a] = ++id;
    ccsz++;
    st.push_back(a);
    for (auto &elm : adj[a]) {
        if (elm == par) continue;
        if (tin[elm]) {
            low[a] = min(low[a], tin[elm]);
        }
    }
}

```

```

        continue;
    }
    dfs(elm, a);
    low[a] = min(low[a], low[elm]);

    if(low[elm] >= tin[a]){
        cc++;
        bcadj[a].push_back(cc);
        bcadj[cc].push_back(a);
        // cout << cc << '\n';
        while(bcadj[cc].empty() || bcadj[cc].back() !=
            elm){
            bcadj[cc].push_back(st.back());
            bcadj[st.back()].push_back(cc);
            st.pop_back();
        }
    }
}

void bctree(int a, int par){
    sz[a] = (a <= n);
    for(auto &elm: bcadj[a]){
        if(elm == par)continue;
        bctree(elm, a);
        sz[a] += sz[elm];
        if(a > n){
            //a = bcc
            ans -= (bcadj[a].size() - 1) * sz[elm] *
                (sz[elm] - 1);
        }
    }
    if(a > n){
        int num = ccsz - sz[a];
        ans -= num * (num - 1) * (bcadj[a].size() - 1);
    }
}

void solve(){
    cin >> n >> m;
    cc = n;
    for(int i=1; i<=m; ++i){
        int a, b;cin >> a >> b;
        adj[a].push_back(b);
        adj[b].push_back(a);
    }
    for(int i=1; i<=n; ++i){
        if(!tin[i]){
            ccsz = 0;
            dfs(i, -1);
            ans += ccsz * (ccsz - 1) * (ccsz - 2);
            bctree(i, -1);
        }
    }
    cout << ans << '\n';
}

```

5.3 Dinic

```

// 0(m * min(n ^ (2 / 3), m ^ (1 / 2))) neu moi c = 1
// 0(m * n * n)

```

```

struct Dinic
{
    struct edge{
        int x, y;
        ll c, f;
        edge(int a = 0, int b = 0, ll z = 0, ll u = 0){
            x = a;
            y = b;
            c = z;
            f = u;
        }
    };

    int n, s, t;
    vector<int> d, pt;
    vector<edge> e;
    vector<vector<int>> adj;

    void init(int len, int a, int b){
        s = a;
        t = b;
        n = len;
        adj.assign(n + 1, vector<int> ());
        d.assign(n + 1, 0);
        pt.assign(n + 1, 0);
        e.clear();
    }

    void adde(int a, int b, ll c){
        adj[a].push_back(e.size());
        e.push_back(edge(a, b, c, 0));
        adj[b].push_back(e.size());
        e.push_back(edge(b, a, 0, 0));
    }

    int bfs(){
        for(int i=1; i<=n; ++i) d[i] = 0;
        deque<int> bfs;
        bfs.push_back(s);
        d[s] = 1;
        while(!bfs.empty()){
            int x = bfs.front();
            bfs.pop_front();
            for(auto &id: adj[x]){
                edge& elm = e[id];
                int y = elm.y;
                if(d[y] != 0 || elm.c == elm.f) continue;
                d[y] = d[x] + 1;
                bfs.push_back(y);
            }
        }
        return d[t];
    }

    ll dfs(int a, ll val){
        if(a == t || val == 0) return val;
        for(; pt[a] < adj[a].size(); pt[a]++){
            int id = adj[a][pt[a]];
            int b = e[id].y;

            if(d[b] != d[a] + 1) continue;
            if(e[id].c == e[id].f) continue;

            ll nxt = dfs(b, min(val, e[id].c - e[id].f));
            if(nxt == 0) continue;

```

```

            e[id].f += nxt;
            e[id ^ 1].f -= nxt;
            return nxt;
        }
        return 0;
    }

    ll maxflow(){
        ll ans = 0;
        while(bfs()){
            for(int i=1; i<=n; ++i) pt[i] = 0;
            while(ll val = dfs(s, INF)) ans += val;
        }
        return ans;
    }
};

int n, m, s, t;
Dinic fl;

void solve(){
    cin >> n >> m >> s >> t;
    fl.init(n, s, t);
    for(int i=1; i<=m; ++i){
        int a, b;ll c;
        cin >> a >> b >> c;
        fl.adde(a, b, c);
    }
    cout << fl.maxflow() << '\n';
}

```

5.4 Edmonds-Karp

```

const int maxn = 1010;
ll ans;
int n, m, s, t;
ll c[maxn][maxn], f[maxn][maxn], trace[maxn];
vector<int> adj[maxn];

void find_path(){
    for(int i=1; i<=n; ++i) trace[i] = 0;
    vector<ll> max_cap(n + 1, 0);

    priority_queue<pair<ll, int>> pq;
    pq.push({INF, s});
    max_cap[s] = INF;
    trace[s] = -1;

    while(!pq.empty()){
        auto [cap, x] = pq.top();
        pq.pop();
        if(x == t) break;
        if(cap < max_cap[x]) continue;
        for(auto &elm: adj[x]){
            ll residual = c[x][elm] - f[x][elm];
            if(residual == 0) continue;
            ll new_cap = min(cap, residual);
            if(new_cap > max_cap[elm]){
                max_cap[elm] = new_cap;
                trace[elm] = x;
            }

```

```

        pq.push({new_cap, elm});
    }
}

void updans(){
    if(trace[t] == 0) return;
    int x = t;
    ll minn = INF;
    while(x != s){
        minn = min(minn, c[trace[x]][x] - f[trace[x]][x]);
        x = trace[x];
    }
    ans += minn;
    x = t;
    while(x != s){
        f[trace[x]][x] += minn;
        f[x][trace[x]] -= minn;
        x = trace[x];
    }
}

void solve(){
    cin >> n >> m >> s >> t;
    for(int i=1; i<=m; ++i){
        int x, y, z; cin >> x >> y >> z;
        adj[x].push_back(y);
        adj[y].push_back(x);
        c[x][y] += z;
    }
    ans = 0;
    while(1){
        find_path();
        if(!trace[t]) break;
        updans();
    }
    cout << ans << '\n';
}

```

5.5 JointBridge

```

// Born_To_Laugh - Hughie Do
#include <bits/stdc++.h>
#define alle(Lotus) Lotus.begin(), Lotus.end()
using namespace std;
typedef long long ll;
[[maybe_unused]] const ll MOD = 998244353, INF = 1e9 + 7;
#define int ll
const int maxn = 1e4 + 10;
int n, m;
vector<int> adj[maxn];
int tin[maxn], tout[maxn], low[maxn];
int id = 0;
int cntj = 0, cntb = 0;
void dfs(int a, int par){
    id++;
    tin[a] = id;
    low[a] = id;
    bool ok = 0;

```

```

    int child = 0;
    for(auto &elm: adj[a]){
        if(elm == par) continue;
        if(tin[elm] == 0){
            child++;
            dfs(elm, a);
            low[a] = min(low[a], low[elm]);

            if(low[elm] == tin[elm]) cntb++;

            ok |= (par != -1 && (low[elm] >= tin[a]));
        }
        else low[a] = min(low[a], tin[elm]);
    }
    ok |= (par == -1 && child > 1);
    if(ok) cntj++;
    tout[a] = id;
}

void solve(){
    cin >> n >> m;
    for(int i=1; i<=m; ++i){
        int a, b; cin >> a >> b;
        adj[a].push_back(b);
        adj[b].push_back(a);
    }
    for(int i=1; i<=n; ++i){
        if(!tin[i]) dfs(i, -1);
    }
    cout << cntj << ' ' << cntb;
}

signed main(){
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    solve();
}

```

5.6 MinCostFlow

```

struct MinCostFlow
{
    struct edge
    {
        int x, y;
        ll c, f, w;
    };
    int n, s, t;
    vector<edge> e;
    vector<int> trace;
    vector<vector<int>> adj;
    vector<ll> pi, dist;
    bool havneg = 0;

    void init(int a, int b, int c){
        n = a;
        s = b;
        t = c;
        havneg = 0;
        adj.assign(n + 1, vector<int> ());
        pi.assign(n + 1, 0);
    }

```

```

        trace.assign(n + 1, 0);
        dist.assign(n + 1, 0);
        e.clear();
    }

    void adde(int x, int y, int c, int w){
        adj[x].push_back(e.size());
        e.push_back({x, y, c, 0, w});
        adj[y].push_back(e.size());
        e.push_back({y, x, 0, 0, -w});
        havneg |= (w < 0);
    }

    void setpi(){
        for(int i=1; i<=n; ++i) pi[i] = INF;
        pi[s] = 0;
        for(int i=1; i<=n; ++i){
            bool upd = 0;
            for(auto &elm: e){
                if(elm.c == elm.f) continue;
                if(pi[elm.x] == INF) continue;
                if(pi[elm.y] > pi[elm.x] + elm.w){
                    pi[elm.y] = pi[elm.x] + elm.w;
                    upd = 1;
                }
            }
            if(!upd) break;
            assert(i != n);
        }
    }

    void dijkstra(){
        for(int i=1; i<=n; ++i) dist[i] = INF;
        for(int i=1; i<=n; ++i) trace[i] = -1;
        priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<>> pq;
        dist[s] = 0;
        pq.push({0, s});
        while(!pq.empty()){
            int x = pq.top().se;
            ll d = pq.top().fi;
            pq.pop();
            if(d > dist[x]) continue;
            for(auto &id: adj[x]){
                edge &elm = e[id];
                if(elm.c == elm.f) continue;

                ll wei = elm.w + pi[elm.x] - pi[elm.y];
                if(dist[elm.y] > dist[elm.x] + wei){
                    dist[elm.y] = dist[elm.x] + wei;
                    trace[elm.y] = id;
                    pq.push({dist[elm.y], elm.y});
                }
            }
        }
    }

    pair<ll, ll> mcfow(ll lim){
        //mincost, fl with fl <=
        lim
        if(havneg) setpi();
        pair<ll, ll> ans = {0, 0};
        while(lim){
            dijkstra();
            if(trace[t] == -1) break;
            ll f = lim;

```

```

int y = t;
while(y != s){
    int id = trace[y];
    f = min(f, e[id].c - e[id].f);
    y = e[id].x;
}
y = t;
while(y != s){
    int id = trace[y];
    ans.fi += e[id].w * f;
    e[id].f += f;
    e[id ^ 1].f -= f;
    y = e[id].x;
}
ans.se += f;
lim -= f;
for(int i=1; i<=n; ++i) pi[i] = min(pi[i] +
    dist[i], INF);
}
return ans;
}
};

int n, m, s, t;
MinCostFlow fl;

void solve(){
    cin >> n >> m >> s >> t;
    fl.init(n, s, t);
    for(int i=1; i<=m; ++i){
        int x, y, c, w;
        cin >> x >> y >> c >> w;
        fl.adde(x, y, c, w);
    }
    int cc; cin >> cc;
    cout << fl.mcfow(cc).fi << '\n'; // query mincost for
    min(maxflow, cc) flow
}

```

5.7 SCC

```

// Born_To_Laugh - Hughie Do
#include <bits/stdc++.h>
#define alle(AC) AC.begin(), AC.end()
using namespace std;
typedef long long ll;
[[maybe_unused]] const ll MOD = 998244353, INF = 1e9 + 7;

const int maxn = 1e5 + 10;
int n, m;
vector<int> adj[maxn];
vector<int> scadj[maxn];
int del[maxn], tin[maxn], low[maxn];
int num[maxn];
ll comsum[maxn], dp[maxn];
int incom[maxn];
int cc = 0;
int id = 0;
vector<int> st, topo;

```

```

void dfs(int a){
    tin[a] = low[a] = ++id;
    st.push_back(a);
    for(auto &elm: adj[a]){
        if(del[elm]) continue;
        if(tin[elm]){
            low[a] = min(low[a], tin[elm]);
            continue;
        }
        dfs(elm);
        low[a] = min(low[a], low[elm]);
    }
    if(low[a] == tin[a]){
        cc++;
        while(!del[a]){
            del[st.back()] = 1;
            incom[st.back()] = cc;
            comsum[cc] += num[st.back()];
            st.pop_back();
        }
    }
}

void dfstopo(int a){
    // cout << a << ' ' ;
    del[a] = 1;
    for(auto &elm: scadj[a]){
        if(del[elm]) continue;
        // cout << a << ' ' << elm << "\n";
        dfstopo(elm);
    }
    // cout << "\n\n";
    topo.push_back(a);
}

void solve(){
    cin >> n >> m;
    for(int i=1; i<=n; ++i) cin >> num[i];
    for(int i=1; i<=m; ++i){
        int a, b; cin >> a >> b;
        adj[a].push_back(b);
    }
    for(int i=1; i<=n; ++i){
        if(!tin[i]) dfs(i);
    }
    for(int i=1; i<=n; ++i){
        for(auto &elm: adj[i]){
            if(incom[i] != incom[elm]){
                scadj[incom[i]].push_back(incom[elm]);
            }
        }
        del[i] = 0;
    }

    for(int i=1; i<=cc; ++i){
        if(!del[i]) dfstopo(i);
    }
    reverse(alle(topo));
    ll ans = 0;
    for(auto &i: topo){
        dp[i] = max(dp[i], comsum[i]);
        for(auto &elm: scadj[i]){
            dp[elm] = max(dp[elm], dp[i] + comsum[elm]);
        }
    }
}

```

```

    ans = max(ans, dp[i]);
}
cout << ans << '\n';
}

signed main(){
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    solve();
}

```

5.8 dijkstra

```

struct edge {
    int to;
    long long w;
    edge () {}
    edge (int a, long long b) : to(a), w(b) {}
    bool operator < (const edge &o) const {
        return w > o.w;
    }
};

typedef vector<vector<edge>> graph;

const long long inf = 1000000LL * 100000000LL;

pair<vector<int>, vector<long long>> dijkstra(graph &g,
    int start) {
    int n = g.size();
    vector<long long> d(n, inf);
    vector<int> p(n, -1);
    d[start] = 0;
    priority_queue<edge> q;
    q.push(edge(start, 0));

    while (!q.empty()) {
        int node = q.top().to;
        long long dist = q.top().w;
        q.pop();

        if (dist > d[node]) continue;

        for (int i = 0; i < (int)g[node].size(); i++) {
            int to = g[node][i].to;
            long long w_extra = g[node][i].w;

            if (dist + w_extra < d[to]) {
                p[to] = node;
                d[to] = dist + w_extra;
                q.push(edge(to, d[to]));
            }
        }
    }

    return {p, d};
}

```

5.9 directed mst

```
const int inf = 1000000 + 10;

struct edge {
    int u, v, w;
    edge() {}
    edge(int a, int b, int c) : u(a), v(b), w(c) {}
};

/**
 * Computes the minimum spanning tree for a directed graph
 * - edges : Graph description in the form of list of
 *           edges.
 *   each edge is: From node u to node v with cost w
 * - root : Id of the node to start the DMST.
 * - n    : Number of nodes in the graph.
 */

int dmst(vector<edge> &edges, int root, int n) {
    int ans = 0;
    int cur_nodes = n;
    while (true) {
        vector<int> lo(cur_nodes, inf), pi(cur_nodes, inf);
        for (int i = 0; i < edges.size(); ++i) {
            int u = edges[i].u, v = edges[i].v, w = edges[i].w;
            if (w < lo[v] and u != v) {
                lo[v] = w;
                pi[v] = u;
            }
        }

        lo[root] = 0;
        for (int i = 0; i < lo.size(); ++i) {
            if (i == root) continue;
            if (lo[i] == inf) return -1;
        }
        int cur_id = 0;
        vector<int> id(cur_nodes, -1), mark(cur_nodes, -1);
        for (int i = 0; i < cur_nodes; ++i) {
            ans += lo[i];
            int u = i;
            while (u != root and id[u] < 0 and mark[u] != i) {
                mark[u] = i;
                u = pi[u];
            }
            if (u != root and id[u] < 0) { // Cycle
                for (int v = pi[u]; v != u; v = pi[v])
                    id[v] = cur_id;
                id[u] = cur_id++;
            }
        }

        if (cur_id == 0)
            break;

        for (int i = 0; i < cur_nodes; ++i)
            if (id[i] < 0) id[i] = cur_id++;

        for (int i = 0; i < edges.size(); ++i) {
            int u = edges[i].u, v = edges[i].v, w = edges[i].w;
```

```
            edges[i].u = id[u];
            edges[i].v = id[v];
            if (id[u] != id[v])
                edges[i].w -= lo[v];
        }
        cur_nodes = cur_id;
        root = id[root];
    }

    return ans;
}
```

5.10 eulerian path

```
// Taken from
// https://github.com/lbv/pc-code/blob/master/code/graph.cpp
// Eulerian Trail

struct Euler {
    ELV adj; IV t;
    Euler(ELV Adj) : adj(Adj) {}
    void build(int u) {
        while(! adj[u].empty()) {
            int v = adj[u].front().v;
            adj[u].erase(adj[u].begin());
            build(v);
        }
        t.push_back(u);
    }
};

bool eulerian_trail(IV &trail) {
    Euler e(adj);
    int odd = 0, s = 0;
    /*
     * for (int v = 0; v < n; v++) {
     *   int diff = abs(in[v] - out[v]);
     *   if (diff > 1) return false;
     *   if (diff == 1) {
     *     if (++odd > 2) return false;
     *     if (out[v] > in[v]) start = v;
     *   }
     * }
     */
    e.build(s);
    reverse(e.t.begin(), e.t.end());
    trail = e.t;
    return true;
}
```

5.11 karp min mean cycle

```
/**
 * Finds the min mean cycle, if you need the max mean cycle
 * just add all the edges with negative cost and print
 * ans * -1
 */
```

```
* test: uva, 11090 - Going in Cycle!!
*/

const int MN = 1000;
struct edge{
    int v;
    long long w;
    edge(){} edge(int v, int w) : v(v), w(w) {}
};

long long d[MN][MN];
// This is a copy of g because increments the size
// pass as reference if this does not matter.
int karp(vector<vector<edge> > g) {
    int n = g.size();

    g.resize(n + 1); // this is important

    for (int i = 0; i < n; ++i)
        if (!g[i].empty())
            g[n].push_back(edge(i, 0));
    ++n;

    for (int i = 0; i < n; ++i)
        fill(d[i], d[i] + (n + 1), INT_MAX);

    d[n - 1][0] = 0;

    for (int k = 1; k <= n; ++k) for (int u = 0; u < n; ++u)
        {
            if (d[u][k - 1] == INT_MAX) continue;
            for (int i = g[u].size() - 1; i >= 0; --i)
                d[g[u][i].v][k] = min(d[g[u][i].v][k], d[u][k - 1] +
                    g[u][i].w);
        }

    bool flag = true;

    for (int i = 0; i < n && flag; ++i)
        if (d[i][n] != INT_MAX)
            flag = false;

    if (flag) {
        return true; // return true if there is no a cycle.
    }

    double ans = 1e15;

    for (int u = 0; u + 1 < n; ++u) {
        if (d[u][n] == INT_MAX) continue;
        double W = -1e15;

        for (int k = 0; k < n; ++k)
            if (d[u][k] != INT_MAX)
                W = max(W, (double)(d[u][n] - d[u][k]) / (n - k));

        ans = min(ans, W);
    }

    // printf("%.2lf\n", ans);
    cout << fixed << setprecision(2) << ans << endl;
```

```
    return false;
}
```

5.12 magic max flow

```
const int N = 1000 + 5;
const int MAXA = 1e9;

int a[N][N];
int n, m, p[N];
bool f[N];

int bfs(){
    memset(f, 0, sizeof f);
    memset(p, 0, sizeof p);
    f[1] = true;
    queue<pair<int, int>> q;
    q.push({1, MAXA});
    while (!q.empty()){
        pair<int, int> st = q.front(); q.pop();
        int u = st.first, val = st.second;

        for(int v = 1; v <= n; ++v){
            if (!f[v] && a[u][v]){
                q.push({v, min(val, a[u][v])});
                f[v] = true; p[v] = u;
            }
        }
        if (v == n) return min(val, a[u][v]);
    }
    return 0;
}

int max_flow(){
    int ans = 0, flow = 0;
    while ((flow = bfs())){
        ans += flow;
        int u = n;
        while (u != 1){
            a[p[u]][u] -= flow;
            a[u][p[u]] += flow;
            u = p[u];
        }
    }
    return ans;
}

void solve()
{
    cin >> n >> m;
    for(int i = 1; i <= m; ++i){
        int u, v, w; cin >> u >> v >> w;
        a[u][v] += w;
    }

    cout << max_flow();
}
```

5.13 maximum matching

```
const int N = 100100;

vector<int> G[N];
int seen[N], iteration = 0;
int dist[N], matchL[N], matchR[N];

bool dfs(int u) {
    if(seen[u] == iteration) return 0;
    seen[u] = iteration;

    for(int v : G[u])
        if(matchR[v] == -1) {
            matchL[u] = v; matchR[v] = u;
            return 1;
        }

    for(int v : G[u])
        if(dist[matchR[v]] == dist[u] + 1 &&
            dfs(matchR[v])) {
            matchL[u] = v; matchR[v] = u;
            return 1;
        }

    return 0;
}

int matching(int M, int N, vector<pair<int, int>> E) {
    fill(matchL + 1, matchL + M + 1, -1);
    fill(matchR + 1, matchR + N + 1, -1);

    for(int i = 0; i < (int) E.size(); ++i) {
        int u = E[i].first, v = E[i].second;
        G[u].push_back(v);
    }

    int ans = 0;
    while(true) {
        queue<int> q;
        for(int u = 1; u <= M; ++u) {
            if(matchL[u] == -1) {
                dist[u] = 0; q.push(u);
            } else {
                dist[u] = -1;
            }
        }
        while(q.size()) {
            int u = q.front(); q.pop();
            for(int v : G[u]) {
                if(matchR[v] != -1 && dist[matchR[v]] == -1)
                    dist[matchR[v]] = dist[u] + 1;
                q.push(matchR[v]);
            }
        }

        int newMatches = 0;
        ++iteration;
        for(int u = 1; u <= M; ++u) {
```

```
            if(matchL[u] == -1) newMatches += dfs(u);
        }
        if(newMatches == 0) break;
        ans += newMatches;
    }

    return ans;
}
```

5.14 minimum path cover in DAG

Given a directed acyclic graph $G = (V, E)$, we are to find the minimum number of vertex-disjoint paths to cover each vertex in V .

We can construct a bipartite graph $G' = (V_{out} \cup V_{in}, E')$ from G , where :

$$V_{out} = \{v \in V : v \text{ has positive out-degree}\}$$

$$V_{in} = \{v \in V : v \text{ has positive in-degree}\}$$

$$E' = \{(u, v) \in V_{out} \times V_{in} : (u, v) \in E\}$$

Then it can be shown, via König's theorem, that G' has a matching of size m if and only if there exists $n - m$ vertex-disjoint paths that cover each vertex in G , where n is the number of vertices in G and m is the maximum cardinality bipartite matching in G' .

Therefore, the problem can be solved by finding the maximum cardinality matching in G' instead.

NOTE: If the paths are not necessarily disjoint, find the transitive closure and solve the problem for disjoint paths.

5.15 query with lca

```
struct lowest_ca {
    int T[MN], L[MN], W[MN];
    int P[MN][ML], MI[MN][ML], MA[MN][ML];

    void dfs(vector<vector<edge>> &g, int root, int pi = -1) {
        if (pi == -1) {
            L[root] = W[root] = 0;
            T[root] = -1;
        }
        for (int i = 0; i < (int)g[root].size(); ++i) {
            int to = g[root][i].v;
            if (to != pi) {
                T[to] = root;
                W[to] = g[root][i].w;
                L[to] = L[root] + 1;
                dfs(g, to, root);
            }
        }
    }
}
```

```

}

void init(vector<vector<edge>> &g, int root) {
    // g is undirected
    dfs(g, root);
    int N = g.size(), i, j;

    for (i = 0; i < N; i++) {
        for (j = 0; j < N; j++) {
            P[i][j] = -1;
            MI[i][j] = inf;
        }
    }

    for (i = 0; i < N; i++) {
        P[i][0] = T[i];
        MI[i][0] = W[i];
    }

    for (j = 1; j < N; j++)
        for (i = 0; i < N; i++)
            if (P[i][j - 1] != -1) {
                P[i][j] = P[P[i][j - 1]][j - 1];
                MI[i][j] = min(MI[i][j - 1], MI[P[i][j - 1]][j - 1]);
            }
    }

    int query(int p, int q) {
        int tmp, log, i;

        int mmin = inf;
        if (L[p] < L[q])
            tmp = p, p = q, q = tmp;

        for (log = 1; 1 << log <= L[p]; log++)
            log--;

        for (i = log; i >= 0; i--)
            if (L[p] - (1 << i) >= L[q]) {
                mmin = min(mmin, MI[p][i]);
                p = P[p][i];
            }

        if (p == q)
            // return p;
            return mmin;

        for (i = log; i >= 0; i--)
            if (P[p][i] != -1 && P[q][i] != P[q][i]) {
                mmin = min(mmin, min(MI[p][i], MI[q][i]));
                p = P[p][i], q = P[q][i];
            }

        // return T[p];
        return min(mmin, min(MI[p][0], MI[q][0]));
    }

    int get_child(int p, int q) { // p is ancestor of q
        if (p == q) return -1;

        int i, log;

```

```

        for (log = 1; 1 << log <= L[q]; log++) {}
        log--;

        for (i = log; i >= 0; i--)
            if (L[q] - (1 << i) > L[p]) {
                q = P[q][i];
            }

        assert(P[q][0] == p);
        return q;
    }

    int is_ancestor(int p, int q) {
        if (L[p] >= L[q])
            return false;

        int dist = L[q] - L[p];

        int cur = q;
        int step = 0;
        while (dist) {
            if (dist & 1)
                cur = P[cur][step];
            step++;
            dist >>= 1;
        }

        return cur == p;
    }
};

```

5.16 two sat (with kosaraju)

```

/**
 * Given a set of clauses (a1 v a2)^(a2 v a3)....
 * this algorithm find a solution to it set of clauses.
 * test:
 *      http://lightoj.com/volume_showproblem.php?problem=1251
 */

#include<bits/stdc++.h>
using namespace std;
#define MAX 100000
#define endl '\n'

vector<int> G[MAX];
vector<int> GT[MAX];
vector<int> Ftime;
vector<vector<int>> SCC;
bool visited[MAX];
int n;

void dfs1(int n){
    visited[n] = 1;

    for (int i = 0; i < G[n].size(); ++i) {
        int curr = G[n][i];
        if (visited[curr]) continue;

```

```

        dfs1(curr);
    }

    Ftime.push_back(n);
}

void dfs2(int n, vector<int> &scc) {
    visited[n] = 1;
    scc.push_back(n);

    for (int i = 0; i < GT[n].size(); ++i) {
        int curr = GT[n][i];
        if (visited[curr]) continue;
        dfs2(curr, scc);
    }
}

void kosaraju() {
    memset(visited, 0, sizeof visited);

    for (int i = 0; i < 2 * n; ++i) {
        if (!visited[i]) dfs1(i);
    }

    memset(visited, 0, sizeof visited);
    for (int i = Ftime.size() - 1; i >= 0; i--) {
        if (visited[Ftime[i]]) continue;
        vector<int> _scc;
        dfs2(Ftime[i], _scc);
        SCC.push_back(_scc);
    }
}

/**
 * After having the SCC, we must traverse each scc, if in
 * one SCC are -b y b, there is not a solution.
 * Otherwise we build a solution, making the first "node"
 * that we find truth and its complement false.
 */

bool two_sat(vector<int> &val) {
    kosaraju();
    for (int i = 0; i < SCC.size(); ++i) {
        vector<bool> tmpvisited(2 * n, false);
        for (int j = 0; j < SCC[i].size(); ++j) {
            if (tmpvisited[SCC[i][j] ^ 1]) return 0;
            if (val[SCC[i][j]] != -1) continue;
            else {
                val[SCC[i][j]] = 0;
                val[SCC[i][j] ^ 1] = 1;
            }
            tmpvisited[SCC[i][j]] = 1;
        }
    }
    return 1;
}

// Example of use

int main() {

```



```

int m, u, v, nc = 0, t; cin >> t;
// n = "nodes" number, m = clauses number

while (t--) {
    cin >> m >> n;
    Ftime.clear();
    SCC.clear();
    for (int i = 0; i < 2 * n; ++i) {
        G[i].clear();
        GT[i].clear();
    }

    // (a1 v a2) = (a1 -> a2) = (a2 -> a1)
    for (int i = 0; i < m; ++i) {
        cin >> u >> v;
        int t1 = abs(u) - 1;
        int t2 = abs(v) - 1;
        int p = t1 * 2 + ((u < 0)? 1 : 0);
        int q = t2 * 2 + ((v < 0)? 1 : 0);
        G[p ^ 1].push_back(q);
        G[q ^ 1].push_back(p);
        GT[p].push_back(q ^ 1);
        GT[q].push_back(p ^ 1);
    }

    vector<int> val(2 * n, -1);
    cout << "Case " << ++nc << ": ";
    if (two_sat(val)) {
        cout << "Yes" << endl;
        vector<int> sol;
        for (int i = 0; i < 2 * n; ++i)
            if (i % 2 == 0 and val[i] == 1)
                sol.push_back(i / 2 + 1);
        cout << sol.size();

        for (int i = 0; i < sol.size(); ++i) {
            cout << " " << sol[i];
        }
        cout << endl;
    } else {
        cout << "No" << endl;
    }
}
return 0;
}

```

6 Math

6.1 Lucas theorem

For non-negative integers m and n and a prime p , the following congruence relation holds :

$$\binom{m}{n} \equiv \prod_{i=0}^k \binom{m_i}{n_i} \pmod{p},$$

where :

$$m = m_k p^k + m_{k-1} p^{k-1} + \dots + m_1 p + m_0,$$

and :

$$n = n_k p^k + n_{k-1} p^{k-1} + \dots + n_1 p + n_0$$

are the base p expansions of m and n respectively. This uses the convention that $\binom{m}{n} = 0$ if $m \leq n$.

6.2 cumulative sum of divisors

```

/**
SOD(n) = sum of actual divisors, CSOD(n) = sod(1) + sod(2)
+ ... + sod(n)
*/

long long csod(long long n) {
    long long ans = 0;
    for (long long i = 2; i * i <= n; ++i) {
        long long j = n / i;
        ans += (i + j) * (j - i + 1) / 2;
        ans += i * (j - i);
    }
    return ans;
}

```

6.3 fibonacci properties

Let A , B and n be integer numbers.

$$k = A - B \quad (1)$$

$$F_A F_B = F_{k+1} F_A^2 + F_k F_A F_{A-1} \quad (2)$$

$$\sum_{i=0}^n F_i^2 = F_{n+1} F_n \quad (3)$$

$ev(n)$ = returns 1 if n is even.

$$\sum_{i=0}^n F_i F_{i+1} = F_{n+1}^2 - ev(n) \quad (4)$$

$$\sum_{i=0}^n F_i F_{i-1} = \sum_{i=0}^{n-1} F_i F_{i+1} \quad (5)$$

6.4 sigma function

the sigma function is defined as:

$$\sigma_x(n) = \sum_{d|n} d^x$$

when $x = 0$ is called the divisor function, that counts the number of positive divisors of n .

Now, we are interested in find

$$\sum_{d|n} \sigma_0(d)$$

if n is written as prime factorization:

$$n = \prod_{i=1}^k P_i^{e_i}$$

we can demonstrate that:

$$\sum_{d|n} \sigma_0(d) = \prod_{i=1}^k g(e_i + 1)$$

where $g(x)$ is the sum of the first x positive numbers:

$$g(x) = (x * (x + 1)) / 2$$

6.5 system of equations

```

const double EPS = 1e-9;
const int INF = 2; // it doesn't actually have to be
infinity or a big number

int gauss (vector < vector<double> > a, vector<double> &
ans) {
    int n = (int) a.size();
    int m = (int) a[0].size() - 1;

    vector<int> where (m, -1);
    for (int col=0, row=0; col<m && row<n; ++col) {
        int sel = row;
        for (int i=row; i<n; ++i)
            if (abs (a[i][col]) > abs (a[sel][col]))
                sel = i;
        if (abs (a[sel][col]) < EPS)
            continue;
        for (int i=col; i<m; ++i)
            swap (a[sel][i], a[row][i]);
        where[col] = row;

        for (int i=0; i<n; ++i)
            if (i != row) {
                double c = a[i][col] / a[row][col];
                for (int j=col; j<m; ++j)
                    a[i][j] -= a[row][j] * c;
            }
    }
}

```

```

    ++row;
}

ans.assign(m, 0);
for (int i=0; i<m; ++i)
    if (where[i] != -1)
        ans[i] = a[where[i]][m] / a[where[i]][i];
for (int i=0; i<n; ++i) {
    double sum = 0;
    for (int j=0; j<m; ++j)
        sum += ans[j] * a[i][j];
    if (abs(sum - a[i][m]) > EPS)
        return 0;
}

for (int i=0; i<m; ++i)
    if (where[i] == -1)
        return INF;
return 1;
}

```

7 Matrix

7.1 MaxMul

```

struct Matrix {
    int n, m;
    vector<vi> d;
    void init(vector<vi> v) {
        d = v;
        n = v.size();
        m = v[0].size();
    }

    void I(int n) {
        vector<vi> v(n, vi(n, 0));
        for (int i = 0; i < n; i++) v[i][i] = 1;
        init(v);
    }

    Matrix operator*(Matrix &other) {
        Matrix res;
        res.n = n;
        res.m = other.m;
        res.d.resize(n, vi(m, 0));
        for (int i = 0; i < res.n; i++) {
            for (int j = 0; j < res.m; j++) {
                for (int k = 0; k < other.n; k++) {
                    res.d[i][j] += d[i][k] * other.d[k][j];
                    res.d[i][j] %= MOD;
                }
            }
        }
        return res;
    }

    Matrix operator^(int k) {
        assert(n == m);

```

```

        Matrix res; res.I(n);
        Matrix cur; cur.init(d);
        while(k > 0) {
            if(k & 1) res = res * cur;
            cur = cur * cur;
            k >>= 1;
        }
        return res;
    }
};

```

8 Misc

8.1 dates and bit

```

//
// Time - Leap years
//

// A[i] has the accumulated number of days from months
// previous to i
const int A[13] = { 0, 0, 31, 59, 90, 120, 151, 181, 212,
    243, 273, 304, 334 };
// same as A, but for a leap year
const int B[13] = { 0, 0, 31, 60, 91, 121, 152, 182, 213,
    244, 274, 305, 335 };
// returns number of leap years up to, and including, y
int leap_years(int y) { return y / 4 - y / 100 + y / 400; }
bool is_leap(int y) { return y % 400 == 0 || (y % 4 == 0
    && y % 100 != 0); }
// number of days in blocks of years
const int p400 = 400*365 + leap_years(400);
const int p100 = 100*365 + leap_years(100);
const int p4 = 4*365 + 1;
const int p1 = 365;
int date_to_days(int d, int m, int y)
{
    return (y - 1) * 365 + leap_years(y - 1) + (is_leap(y) ?
        B[m] : A[m]) + d;
}

void days_to_date(int days, int &d, int &m, int &y)
{
    bool top100; // are we in the top 100 years of a 400
    // block?
    bool top4; // are we in the top 4 years of a 100 block?
    bool top1; // are we in the top year of a 4 block?

    y = 1;
    top100 = top4 = top1 = false;

    y += ((days-1) / p400) * 400;
    d = (days-1) % p400 + 1;

    if (d > p100*3) top100 = true, d -= 3*p100, y += 300;
    else y += ((d-1) / p100) * 100, d = (d-1) % p100 + 1;

    if (d > p4*24) top4 = true, d -= 24*p4, y += 24*4;
    else y += ((d-1) / p4) * 4, d = (d-1) % p4 + 1;
}

```

```

    if (d > p1*3) top1 = true, d -= p1*3, y += 3;
    else y += (d-1) / p1, d = (d-1) % p1 + 1;

    const int *ac = top1 && (!top4 || top100) ? B : A;
    for (m = 1; m < 12; ++m) if (d <= ac[m + 1]) break;
    d -= ac[m];
}

// Bit manipulation
/*
 * n | (1 << x) sets the x-th bit
 * n ^ (1 << x) flips the x-th bit
 * n & ~(1 << x) clears the x-th bit
 */
bool is_set(unsigned int number, int x) {
    return (number >> x) & 1;
}

bool isDivisibleByPowerOf2(int n, int k) {
    int powerOf2 = 1 << k;
    return (n & (powerOf2 - 1)) == 0;
}

bool isPowerOfTwo(unsigned int n) {
    return n && !(n & (n - 1));
}

```

9 Number theory

9.1 Factorizer

```

typedef long long ll;
typedef unsigned long long ull;

struct Factorizer
{
    ull mul_mod(ull a, ull b, ull m){
        ull res = 0;
        a %= m;
        while(b > 0){
            if(b & 1) res = (res + a) % m;
            a = (a + a) % m;
            b >>= 1;
        }
        return res;
    }

    ull modpow(ull base, ull exp, ull mod){
        ull res = 1;
        base %= mod;
        while(exp > 0){
            if(exp & 1) res = mul_mod(res, base, mod);
            base = mul_mod(base, base, mod);
            exp >>= 1;
        }
        return res;
    }

    bool is_prime(ull n){

```

```

if(n < 2) return false;
if(n == 2 || n == 3) return true;
if(n % 2 == 0) return false;
ull d = n - 1;
int s = 0;
while((d & 1) == 0){
    d >>= 1;
    s++;
}
static const ull bases[] = {2, 3, 5, 7, 11, 13, 17,
    19, 23, 29, 31, 37};
for(ull a : bases){
    if(n <= a) break;
    ull x = modpow(a, d, n);
    if(x == 1 || x == n - 1) continue;
    bool composite = true;
    for(int r = 1; r < s; r++){
        x = mul_mod(x, x, n);
        if(x == n - 1){
            composite = false;
            break;
        }
    }
    if(composite) return false;
}
return true;
}
ull pollard_rho(ull n){
    if(n % 2 == 0) return 2;
    if(is_prime(n)) return n;
    ull x = 2, y = 2, d = 1, c = 1;
    auto f = [&](ull x, ull n, ull c){
        return (mul_mod(x, x, n) + c) % n;
    };
    while(d == 1){
        x = f(x, n, c);
        y = f(f(y, n, c), n, c);
        ull diff = (x > y) ? (x - y) : (y - x);
        d = gcd(diff, n);
        if(d == n){
            x = rand() % (n - 2) + 2;
            y = x;
            c = rand() % (n - 1) + 1;
            d = 1;
        }
    }
    return d;
}
void factorize_helper(ull n, vector<ull>& primes){
    if(n <= 1) return;
    if(is_prime(n)){
        primes.push_back(n);
        return;
    }
    ull divisor = pollard_rho(n);
    factorize_helper(divisor, primes);
    factorize_helper(n / divisor, primes);
}
vector<pair<ll, int>> get_factors(ll n){
    vector<ull> primes;
    factorize_helper(n, primes);
    sort(all(primes));

```

```

vector<pair<ll, int>> factors;
for(ull p : primes){
    if(!factors.empty() && factors.back().fi ==
        (ll)p){
        factors.back().se++;
    } else {
        factors.push_back({(ll)p, 1});
    }
}
return factors;
}
};

// Usage: Factorizer fact; vector<pair<ll, int>>
fact.get_factors(n)

```

9.2 crt

```

/**
 * Chinese remainder theorem.
 * Find z such that z % x[i] = a[i] for all i.
 */
long long crt(vector<long long> &a, vector<long long> &x) {
    long long z = 0;
    long long n = 1;
    for (int i = 0; i < x.size(); ++i)
        n *= x[i];

    for (int i = 0; i < a.size(); ++i) {
        long long tmp = (a[i] * (n / x[i])) % n;
        tmp = (tmp * mod_inv(n / x[i], x[i])) % n;
        z = (z + tmp) % n;
    }

    return (z + n) % n;
}

```

9.3 diophantine equations

```

long long gcd(long long a, long long b, long long &x, long
    long &y) {
    if (a == 0) {
        x = 0;
        y = 1;
        return b;
    }
    long long x1, y1;
    long long d = gcd(b % a, a, x1, y1);
    x = y1 - (b / a) * x1;
    y = x1;
    return d;
}

bool find_any_solution(long long a, long long b, long long
    c, long long &x0,
    long long &y0, long long &g) {

```

```

g = gcd(abs(a), abs(b), x0, y0);
if (c % g) {
    return false;
}

x0 *= c / g;
y0 *= c / g;
if (a < 0) x0 = -x0;
if (b < 0) y0 = -y0;
return true;
}

void shift_solution(long long &x, long long &y, long long
    a, long long b,
    long long cnt) {
    x += cnt * b;
    y -= cnt * a;
}

long long find_all_solutions(long long a, long long b,
    long long c,
    long long minx, long long maxx, long long miny,
    long long maxy) {
    long long x, y, g;
    if (!find_any_solution(a, b, c, x, y, g)) return 0;
    a /= g;
    b /= g;

    long long sign_a = a > 0 ? +1 : -1;
    long long sign_b = b > 0 ? +1 : -1;

    shift_solution(x, y, a, b, (minx - x) / b);
    if (x < minx) shift_solution(x, y, a, b, sign_b);
    if (x > maxx) return 0;
    long long lx1 = x;

    shift_solution(x, y, a, b, (maxx - x) / b);
    if (x > maxx) shift_solution(x, y, a, b, -sign_b);
    long long rx1 = x;

    shift_solution(x, y, a, b, -(miny - y) / a);
    if (y < miny) shift_solution(x, y, a, b, -sign_a);
    if (y > maxy) return 0;
    long long ly2 = y;

    shift_solution(x, y, a, b, -(maxy - y) / a);
    if (y > maxy) shift_solution(x, y, a, b, sign_a);
    long long rx2 = x;

    if (lx2 > rx2) swap(lx2, rx2);
    long long lx = max(lx1, lx2);
    long long rx = min(rx1, rx2);

    if (lx > rx) return 0;
    return (rx - lx) / abs(b) + 1;
}

```

9.4 discrete logarithm

```
// Computes x which  $a^x \equiv b \pmod n$ .
```

```
long long d_log(long long a, long long b, long long n) {
    long long m = ceil(sqrt(n));
    long long aj = 1;
    map<long long, long long> M;
    for (int i = 0; i < m; ++i) {
        if (!M.count(aj))
            M[aj] = i;
        aj = (aj * a) % n;
    }

    long long coef = mod_pow(a, n - 2, n);
    coef = mod_pow(coef, m, n);
    // coef =  $a^{-(m)}$ 
    long long gamma = b;
    for (int i = 0; i < m; ++i) {
        if (M.count(gamma)) {
            return i * m + M[gamma];
        } else {
            gamma = (gamma * coef) % n;
        }
    }
    return -1;
}
```

9.5 ext euclidean

```
void ext_euclid(long long a, long long b, long long &x,
               long long &y, long long &g) {
    x = 0, y = 1, g = b;
    long long m, n, q, r;
    for (long long u = 1, v = 0; a != 0; g = a, a = r) {
        q = g / a, r = g % a;
        m = x - u * q, n = y - v * q;
        x = u, y = v, u = m, v = n;
    }
}
```

9.6 highest exponent factorial

```
/*
 * Largest k so that n! is divisible by  $p^k$ 
 * If k is prime then use algorithm below
 * If not, we have  $k = k_1!^{p_1} \cdot \dots \cdot k_m!^{p_m}$ 
 * Then the answer will be  $\min(\text{highest\_exponent}(k_i, n) / p_i)$  with  $i = 1 \dots m$ 
 */
int highest_exponent(int p, const int &n){
    int ans = 0;
    int t = p;
    while(t <= n){
        ans += n/t;
        t*=p;
    }
```

```
}
return ans;
}
```

9.7 miller rabin

```
const int rounds = 20;

// checks whether a is a witness that n is not prime,  $1 < a < n$ 
bool witness(long long a, long long n) {
    // check as in Miller Rabin Primality Test described
    long long u = n - 1;
    int t = 0;
    while (u % 2 == 0) {
        t++;
        u >>= 1;
    }
    long long next = mod_pow(a, u, n);
    if (next == 1) return false;
    long long last;
    for (int i = 0; i < t; ++i) {
        last = next;
        next = mod_mul(last, last, n);
        if (next == 1) {
            return last != n - 1;
        }
    }
    return next != 1;
}
```

```
// Checks if a number is prime with prob  $1 - 1 / (2^{it})$ 
// D(miller_rabin(99999999999999997LL)) == 1;
// D(miller_rabin(999999999999999971LL)) == 1;
// D(miller_rabin(7907)) == 1;
bool miller_rabin(long long n, int it = rounds) {
    if (n <= 1) return false;
    if (n == 2) return true;
    if (n % 2 == 0) return false;
    for (int i = 0; i < it; ++i) {
        long long a = rand() % (n - 1) + 1;
        if (witness(a, n)) {
            return false;
        }
    }
    return true;
}
```

9.8 mod integer

```
template<class T, T mod>
struct mint_t {
    T val;
    mint_t() : val(0) {}
    mint_t(T v) : val(v % mod) {}
```

```
mint_t operator + (const mint_t& o) const {
    return (val + o.val) % mod;
}
mint_t operator - (const mint_t& o) const {
    return (val - o.val) % mod;
}
mint_t operator * (const mint_t& o) const {
    return (val * o.val) % mod;
}
};
```

```
typedef mint_t<long long, 998244353> mint;
```

9.9 mod inv

```
long long mod_inv(long long n, long long m) {
    long long x, y, gcd;
    ext_euclid(n, m, x, y, gcd);
    if (gcd != 1)
        return 0;
    return (x + m) % m;
}
```

9.10 mod mul

```
// Computes  $(a * b) \% \text{mod}$ 
long long mod_mul(long long a, long long b, long long mod) {
    {
        long long x = 0, y = a % mod;
        while (b > 0) {
            if (b & 1)
                x = (x + y) % mod;
            y = (y * 2) % mod;
            b /= 2;
        }
        return x % mod;
    }
}
```

9.11 primes

```
namespace primes {
    const int MP = 100001;
    bool sieve[MP];
    long long primes[MP];
    int num_p;
    void fill_sieve() {
        num_p = 0;
        sieve[0] = sieve[1] = true;
        for (long long i = 2; i < MP; ++i) {
            if (!sieve[i]) {
                primes[num_p++] = i;
                for (long long j = i * i; j < MP; j += i)
```

```

        sieve[j] = true;
    }
}

// Finds prime numbers between a and b, using basic
// primes up to sqrt(b)
// a must be greater than 1.
vector<long long> seg_sieve(long long a, long long b) {
    long long ant = a;
    a = max(a, 3LL);
    vector<bool> pmap(b - a + 1);
    long long sqrt_b = sqrt(b);
    for (int i = 0; i < num_p; ++i) {
        long long p = primes[i];
        if (p > sqrt_b) break;
        long long j = (a + p - 1) / p;
        for (long long v = (j == 1) ? p + p : j * p; v <= b;
            v += p) {
            pmap[v - a] = true;
        }
    }
    vector<long long> ans;
    if (ant == 2) ans.push_back(2);
    int start = a % 2 ? 0 : 1;
    for (int i = start, I = b - a + 1; i < I; i += 2)
        if (pmap[i] == false)
            ans.push_back(a + i);
    return ans;
}

vector<pair<int, int>> factor(int n) {
    vector<pair<int, int>> ans;
    if (n == 0) return ans;
    for (int i = 0; primes[i] * primes[i] <= n; ++i) {
        if ((n % primes[i]) == 0) {
            int expo = 0;
            while ((n % primes[i]) == 0) {
                expo++;
                n /= primes[i];
            }
            ans.emplace_back(primes[i], expo);
        }
    }

    if (n > 1) {
        ans.emplace_back(n, 1);
    }
    return ans;
}

```

9.12 totient sieve

```

for (int i = 1; i < MN; i++)
    phi[i] = i;

for (int i = 1; i < MN; i++)
    if (!sieve[i]) // is prime

```

```

for (int j = i; j < MN; j += i)
    phi[j] -= phi[j] / i;

```

9.13 totient

```

long long totient(long long n) {
    if (n == 1) return 0;
    long long ans = n;
    for (int i = 0; primes[i] * primes[i] <= n; ++i) {
        if ((n % primes[i]) == 0) {
            while ((n % primes[i]) == 0) n /= primes[i];
            ans -= ans / primes[i];
        }
    }
    if (n > 1) {
        ans -= ans / n;
    }
    return ans;
}

```

10 Strings

10.1 Incremental Aho Corasick

```

class IncrementalAhoCorasick {
    static const int Alphabets = 26;
    static const int AlphabetBase = 'a';
    struct Node {
        Node *fail;
        Node *next[Alphabets];
        int sum;
        Node() : fail(NULL), next{}, sum(0) {}
    };

    struct String {
        string str;
        int sign;
    };

public:
    //totalLen = sum of (len + 1)
    void init(int totalLen) {
        nodes.resize(totalLen);
        nNodes = 0;
        strings.clear();
        roots.clear();
        sizes.clear();
        que.resize(totalLen);
    }

    void insert(const string &str, int sign) {
        strings.push_back(String{ str, sign });
        roots.push_back(nodes.data() + nNodes);
        sizes.push_back(1);
        nNodes += (int)str.size() + 1;
    }
}

```

```

auto check = [&]() { return sizes.size() > 1 &&
    sizes.end()[-1] == sizes.end()[-2]; };
if(!check())
    makePMA(strings.end() - 1, strings.end(),
        roots.back(), que);
while(check()) {
    int m = sizes.back();
    roots.pop_back();
    sizes.pop_back();
    sizes.back() += m;
    if(!check())
        makePMA(strings.end() - m * 2, strings.end(),
            roots.back(), que);
}
}

```

```

int match(const string &str) const {
    int res = 0;
    for(const Node *t : roots)
        res += matchPMA(t, str);
    return res;
}

```

private:

```

static void makePMA(vector<String>::const_iterator
    begin, vector<String>::const_iterator end, Node
    *nodes, vector<Node*> &que) {
    int nNodes = 0;
    Node *root = new(&nodes[nNodes++]) Node();
    for(auto it = begin; it != end; ++it) {
        Node *t = root;
        for(char c : it->str) {
            Node *&n = t->next[c - AlphabetBase];
            if(n == nullptr)
                n = new(&nodes[nNodes++]) Node();
            t = n;
        }
        t->sum += it->sign;
    }
    int qt = 0;
    for(Node *&n : root->next) {
        if(n != nullptr) {
            n->fail = root;
            que[qt++] = n;
        } else {
            n = root;
        }
    }
    for(int qh = 0; qh != qt; ++qh) {
        Node *t = que[qh];
        int a = 0;
        for(Node *n : t->next) {
            if(n != nullptr) {
                que[qt++] = n;
                Node *r = t->fail;
                while(r->next[a] == nullptr)
                    r = r->fail;
                n->fail = r->next[a];
                n->sum += r->next[a]->sum;
            }
            ++a;
        }
    }
}

```

```

    }
}

static int matchPMA(const Node *t, const string &str) {
    int res = 0;
    for(char c : str) {
        int a = c - AlphabetBase;
        while(t->next[a] == nullptr)
            t = t->fail;
        t = t->next[a];
        res += t->sum;
    }
    return res;
}

vector<Node> nodes;
int nNodes;
vector<String> strings;
vector<Node*> roots;
vector<int> sizes;
vector<Node*> que;
};

int main() {
    int m;
    while(~scanf("%d", &m)) {
        IncrementalAhoCorasic iac;
        iac.init(600000);
        rep(i, m) {
            int ty;
            char s[300001];
            scanf("%d%s", &ty, s);
            if(ty == 1) {
                iac.insert(s, +1);
            } else if(ty == 2) {
                iac.insert(s, -1);
            } else if(ty == 3) {
                int ans = iac.match(s);
                printf("%d\n", ans);
                fflush(stdout);
            } else {
                abort();
            }
        }
    }
    return 0;
}

```

10.2 KMP

```

// pi[i] = max(k = 0...i, s[0...k - 1] = s[i - (k - 1)...i])
vector<int> prefix_function(string s) {
    int n = (int)s.length();
    vector<int> pi(n);
    for (int i = 1; i < n; i++) {
        int j = pi[i - 1];
        while (j > 0 && s[i] != s[j])

```

```

        j = pi[j - 1];
        if (s[i] == s[j])
            j++;
        pi[i] = j;
    }
    return pi;
}

```

10.3 ZFunc

```

vector<int> z_function(string s) {
    int n = s.length();
    vector<int> z(n);
    for (int i = 1, l = 0, r = 0; i < n; ++i) {
        if (i <= r) z[i] = min(r - i + 1, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]])
            ++z[i];
        if (i + z[i] - 1 > r) {
            l = i;
            r = i + z[i] - 1;
        }
    }
    return z;
}
/*
 * alfalfa
 * 0 0 0 4 0 0 1
 * z[0] = 0, z[i] = max(k : s[0..k] = s[i...i + k - 1])
 */

```

10.4 manacher

```

/*
 * Longest palindrome substring in O(n)
 */
vector<int> manacher_odd(string s) {
    int n = s.size();
    s = "$" + s + "~";
    vector<int> p(n + 2);
    int l = 1, r = 1;
    for(int i = 1; i <= n; i++) {
        p[i] = max(0, min(r - i, p[l + (r - i)]));
        while(s[i - p[i]] == s[i + p[i]]) {
            p[i]++;
        }
        if(i + p[i] > r) {
            l = i - p[i], r = i + p[i];
        }
    }
    return vector<int>(begin(p) + 1, end(p) - 1);
}

vector<int> manacher(string s) {
    string t;
    for(auto c: s) {
        t += string("#") + c;
    }
}

```

```

    auto res = manacher_odd(t + "#");
    return vector<int>(begin(res) + 1, end(res) - 1);
}

```

10.5 minimal string rotation

```

// Lexicographically minimal string rotation
int lmsr() {
    string s;
    cin >> s;
    int n = s.size();
    s += s;
    vector<int> f(s.size(), -1);
    int k = 0;
    for (int j = 1; j < 2 * n; ++j) {
        int i = f[j - k - 1];
        while (i != -1 && s[j] != s[k + i + 1]) {
            if (s[j] < s[k + i + 1])
                k = j - i - 1;
            i = f[i];
        }
        if (i == -1 && s[j] != s[k + i + 1]) {
            if (s[j] < s[k + i + 1]) {
                k = j;
            }
            f[j - k] = -1;
        } else {
            f[j - k] = i + 1;
        }
    }
    return k;
}

```

10.6 suffix array

```

/**
 * O (n log^2 (n))
 * See
 * http://web.stanford.edu/class/cs97si/suffix-array.pdf
 * for reference
 */

struct entry{
    int a, b, p;
    entry(){}
    entry(int x, int y, int z): a(x), b(y), p(z){}
    bool operator < (const entry &o) const {
        return (a == o.a) ? (b == o.b) ? (p < o.p) : (b < o.b)
            : (a < o.a);
    }
};

struct SuffixArray{
    const int N;
    string s;
    vector<vector<int>> > P;
}

```

```

vector<entry> M;

SuffixArray(const string &s) : N(s.length()) , s(s),
    P(1, vector<int>(N, 0)), M(N) {
    for (int i = 0; i < N; ++i)
        P[0][i] = (int) s[i];

    for (int skip = 1, level = 1; skip < N; skip *= 2,
        level++) {
        P.push_back(vector<int>(N, 0));
        for (int i = 0; i < N; ++i) {
            int next = ((i + skip) < N) ? P[level - 1][i +
                skip] : -10000;
            M[i] = entry(P[level - 1][i], next, i);
        }
        sort(M.begin(), M.end());
        for (int i = 0; i < N; ++i)
            P[level][M[i].p] = (i > 0 and M[i].a == M[i - 1].a
                and M[i].b == M[i - 1].b) ? P[level][M[i -
                    1].p] : i;
    }
}

vector<int> getSuffixArray(){
    vector<int> &rank = P.back();
    vector<pair<int, int>> inv(rank.size());
    for (int i = 0; i < rank.size(); ++i)
        inv[i] = make_pair(rank[i], i);
    sort(inv.begin(), inv.end());
    vector<int> sa(rank.size());
    for (int i = 0; i < rank.size(); ++i)
        sa[i] = inv[i].second;
    return sa;
}

// returns the length of the longest common prefix of
// s[i...L-1] and s[j...L-1]
int lcp(int i, int j) {
    int len = 0;
    if (i == j) return N - i;
    for (int k = P.size() - 1; k >= 0 && i < N && j < N;
        --k) {
        if (P[k][i] == P[k][j]) {
            i += 1 << k;
            j += 1 << k;
            len += 1 << k;
        }
    }
    return len;
}
};

```

10.7 suffix automaton

```

/*
 * Suffix automaton:
 * This implementation was extended to maintain (online)
 * the

```

```

 * number of different substrings. This is equivalent to
 * compute
 * the number of paths from the initial state to all the
 * other
 * states.
 *
 * The overall complexity is O(n)
 * can be tested here:
 * https://www.urionlinejudge.com.br/judge/en/problems/view/1530
 */

```

```

struct state {
    int len, link;
    long long num_paths;
    map<int, int> next;
};

```

```

const int MN = 200011;
state sa[MN << 1];
int sz, last;
long long tot_paths;

```

```

void sa_init() {
    sz = 1;
    last = 0;
    sa[0].len = 0;
    sa[0].link = -1;
    sa[0].next.clear();
    sa[0].num_paths = 1;
    tot_paths = 0;
}

```

```

void sa_extend(int c) {
    int cur = sz++;
    sa[cur].len = sa[last].len + 1;
    sa[cur].next.clear();
    sa[cur].num_paths = 0;
    int p;
    for (p = last; p != -1 && !sa[p].next.count(c); p =
        sa[p].link) {
        sa[p].next[c] = cur;
        sa[cur].num_paths += sa[p].num_paths;
        tot_paths += sa[p].num_paths;
    }
}

```

```

if (p == -1) {
    sa[cur].link = 0;
} else {
    int q = sa[p].next[c];
    if (sa[p].len + 1 == sa[q].len) {
        sa[cur].link = q;
    } else {
        int clone = sz++;
        sa[clone].len = sa[p].len + 1;
        sa[clone].next = sa[q].next;
        sa[clone].num_paths = 0;
        sa[clone].link = sa[q].link;
        for (; p != -1 && sa[p].next[c] == q; p = sa[p].link) {
            sa[p].next[c] = clone;
            sa[q].num_paths -= sa[p].num_paths;
            sa[clone].num_paths += sa[p].num_paths;
        }
    }
}

```

```

    sa[q].link = sa[cur].link = clone;
}
}
last = cur;
}

```

11 X - Extra Math

11.1 equations

$$ax^2 + bx + c = 0 \Rightarrow x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

The extremum is given by $x = -b/2a$.

$$\begin{aligned} ax + by = e &\Rightarrow x = \frac{ed - bf}{ad - bc} \\ cx + dy = f &\Rightarrow y = \frac{af - ec}{ad - bc} \end{aligned}$$

In general, given an equation $Ax = b$, the solution to a variable x_i is given by

$$x_i = \frac{\det A'_i}{\det A}$$

where A'_i is A with the i 'th column replaced by b .

11.2 Trigonometry and Triangles

$$\sin(v + w) = \sin v \cos w + \cos v \sin w$$

$$\cos(v + w) = \cos v \cos w - \sin v \sin w$$

$$\tan(v + w) = \frac{\tan v + \tan w}{1 - \tan v \tan w}$$

$$\sin v + \sin w = 2 \sin \frac{v + w}{2} \cos \frac{v - w}{2}$$

$$\cos v + \cos w = 2 \cos \frac{v + w}{2} \cos \frac{v - w}{2}$$

$$(V + W) \tan(v - w)/2 = (V - W) \tan(v + w)/2$$

where V, W are lengths of sides opposite angles v, w .

$$a \cos x + b \sin x = r \cos(x - \phi)$$

$$a \sin x + b \cos x = r \sin(x + \phi)$$

where $r = \sqrt{a^2 + b^2}$, $\phi = \text{atan2}(b, a)$.

Side lengths: a, b, c

$$\text{Semiperimeter: } p = \frac{a + b + c}{2}$$

$$\text{Area: } A = \sqrt{p(p - a)(p - b)(p - c)}$$

$$\text{Circumradius: } R = \frac{abc}{4A}$$

Inradius: $r = \frac{A}{p}$

Length of median (divides triangle into two equal-area triangles): $m_a = \frac{1}{2}\sqrt{2b^2 + 2c^2 - a^2}$

Length of bisector (divides angles in two): $s_a =$

$$\sqrt{bc \left[1 - \left(\frac{a}{b+c} \right)^2 \right]}$$

Law of sines: $\frac{\sin \alpha}{a} = \frac{\sin \beta}{b} = \frac{\sin \gamma}{c} = \frac{1}{2R}$

Law of cosines: $a^2 = b^2 + c^2 - 2bc \cos \alpha$

Law of tangents: $\frac{a+b}{a-b} = \frac{\tan \frac{\alpha+\beta}{2}}{\tan \frac{\alpha-\beta}{2}}$

11.3 Quadrilaterals

With side lengths a, b, c, d , diagonals e, f , diagonals angle θ , area A and magic flux $F = b^2 + d^2 - a^2 - c^2$:

$$4A = 2ef \cdot \sin \theta = F \tan \theta = \sqrt{4e^2 f^2 - F^2}$$

For cyclic quadrilaterals the sum of opposite angles is 180° , $ef = ac + bd$, and $A = \sqrt{(p-a)(p-b)(p-c)(p-d)}$.

11.4 Spherical coordinates

$$\begin{aligned} x &= r \sin \theta \cos \phi & r &= \sqrt{x^2 + y^2 + z^2} \\ y &= r \sin \theta \sin \phi & \theta &= \arccos(z/\sqrt{x^2 + y^2 + z^2}) \\ z &= r \cos \theta & \phi &= \operatorname{atan2}(y, x) \end{aligned}$$

11.5 Sums and (Geometric) series

$$c^a + c^{a+1} + \dots + c^b = \frac{c^{b+1} - c^a}{c - 1}, c \neq 1$$

$$1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$$

$$1^2 + 2^2 + 3^2 + \dots + n^2 = \frac{n(2n+1)(n+1)}{6}$$

$$1^3 + 2^3 + 3^3 + \dots + n^3 = \frac{n^2(n+1)^2}{4}$$

$$1^4 + 2^4 + 3^4 + \dots + n^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$$

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots, (-\infty < x < \infty)$$

$$\ln(1+x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \dots, (-1 < x \leq 1)$$

$$\sqrt{1+x} = 1 + \frac{x}{2} - \frac{x^2}{8} + \frac{2x^3}{32} - \frac{5x^4}{128} + \dots, (-1 \leq x \leq 1)$$

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots, (-\infty < x < \infty)$$

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \dots, (-\infty < x < \infty)$$

$$r \neq 1$$

$$a + ar + ar^2 + ar^3 + \dots + ar^{n-1} = \sum_{k=0}^{n-1} ar^k = a \left(\frac{1-r^n}{1-r} \right)$$

11.6 Combinatorics

Binomial coefficients

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}, \quad \binom{n}{0} = \binom{n}{n} = 1$$

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k} \quad (\text{Pascal's rule})$$

$$\binom{n}{k} = \binom{n}{n-k}$$

$$\sum_{k=0}^n \binom{n}{k} = 2^n, \quad \sum_{k=0}^n (-1)^k \binom{n}{k} = 0$$

$$\sum_{k=0}^n k \binom{n}{k} = n2^{n-1}, \quad \sum_{k=0}^n k^2 \binom{n}{k} = n(n+1)2^{n-2}$$

$$\binom{r}{k} = \frac{r(r-1)\dots(r-k+1)}{k!}$$

Binomial theorem

$$(x+y)^n = \sum_{k=0}^n \binom{n}{k} x^{n-k} y^k$$

$$(1+x)^r = \sum_{k=0}^{\infty} \binom{r}{k} x^k, \quad |x| < 1$$

Counting formulas

$$P(n, k) = \frac{n!}{(n-k)!} \quad (\text{permutations of } k \text{ from } n)$$

$$C(n, k) = \binom{n}{k} \quad (\text{combinations})$$

$$\text{Combinations with repetition: } \binom{n+k-1}{k}$$

$$\text{Number of subsets of } n \text{ elements: } 2^n$$

Number of ways to divide n items into r groups of size

$$n_1, \dots, n_r : \frac{n!}{n_1! n_2! \dots n_r!}$$

Inclusion–Exclusion

$$|\cup_{i=1}^n A_i| = \sum_{k=1}^n (-1)^{k+1} \sum_{1 \leq i_1 < \dots < i_k \leq n} |A_{i_1} \cap \dots \cap A_{i_k}|$$

Stirling / Bell / Catalan

$$S(n, k) = kS(n-1, k) + S(n-1, k-1)$$

(Stirling 2nd kind: partitions of n items into k non-empty sets)

$$B_n = \sum_{k=0}^n S(n, k) \quad (\text{Bell number})$$

$$C_n = \frac{1}{n+1} \binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1}$$

$$C_0 = 1, \quad C_{n+1} = \sum_{i=0}^n C_i C_{n-i}$$

$$C_n = \frac{(2n)!}{n!(n+1)!}$$

Multinomial theorem

$$(x_1 + x_2 + \dots + x_m)^n = \sum_{k_1 + \dots + k_m = n} \frac{n!}{k_1! k_2! \dots k_m!} x_1^{k_1} x_2^{k_2} \dots x_m^{k_m}$$

Other useful sums

$$\sum_{k=0}^n \binom{r+k}{k} = \binom{r+n+1}{n},$$

$$\sum_{k=0}^n (-1)^k \binom{r}{k} = (-1)^n \binom{r-1}{n}$$

$$\sum_{k=0}^n \binom{a}{k} \binom{b}{n-k} = \binom{a+b}{n} \quad (\text{Vandermonde's identity})$$

$$\sum_{k=0}^n k \binom{r+k}{k} = (n+1) \binom{r+n+1}{n-1}$$

11.7 matrices

Linear recurrences / Companion matrix

Consider the linear recurrence

$$f_n = \sum_{i=1}^k c_i f_{n-i}, \quad n \geq k,$$

with initial values $f_0, \dots, f_{k-1}$. Define the $k \times k$ companion matrix T and the $k \times 1$ column vector F_m by

$$T = \begin{pmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \\ c_k & c_{k-1} & c_{k-2} & \cdots & c_1 \end{pmatrix}, \quad F_m = \begin{pmatrix} f_m \\ f_{m+1} \\ \vdots \\ f_{m+k-1} \end{pmatrix}.$$

11.8 vectors

A vector in R^2 is

$$\vec{a} = (a_x, a_y), \quad |\vec{a}| = \sqrt{a_x^2 + a_y^2}, \quad \hat{a} = \frac{\vec{a}}{|\vec{a}|}.$$

Basic Operations

$$\vec{a} \pm \vec{b} = (a_x \pm b_x, a_y \pm b_y), \quad k\vec{a} = (ka_x, ka_y)$$

Dot Product

$$\vec{a} \cdot \vec{b} = a_x b_x + a_y b_y = |\vec{a}| |\vec{b}| \cos \theta$$

$$\text{proj}_{\vec{b}}(\vec{a}) = \frac{\vec{a} \cdot \vec{b}}{|\vec{b}|^2} \vec{b}, \quad \vec{a} \cdot \vec{b} = 0 \Rightarrow \text{orthogonal}.$$

2D Cross Product (Scalar Form)

$$\text{cross2D}(\vec{a}, \vec{b}) = a_x b_y - a_y b_x$$

$$|\text{cross2D}(\vec{a}, \vec{b})| = |\vec{a}| |\vec{b}| \sin \theta$$

This scalar represents the signed area of the parallelogram formed by $\vec{a}$ and $\vec{b}$.

Areas

$$A_{\text{parallelogram}} = |\text{cross2D}(\vec{a}, \vec{b})|,$$

$$A_{\text{triangle}} = \frac{1}{2} |\text{cross2D}(\vec{a}, \vec{b})|$$

$$A_{\text{polygon}} = \frac{1}{2} \left| \sum_{i=0}^{n-1} (x_i y_{i+1} - y_i x_{i+1}) \right|, \quad p_n = p_0.$$

Centroid of a Polygon

$$C_x = \frac{1}{6A} \sum (x_i + x_{i+1})(x_i y_{i+1} - x_{i+1} y_i),$$

$$C_y = \frac{1}{6A} \sum (y_i + y_{i+1})(x_i y_{i+1} - x_{i+1} y_i)$$

Angle and Rotation

Angle between two vectors:

$$\cos \theta = \frac{\vec{a} \cdot \vec{b}}{|\vec{a}| |\vec{b}|},$$

$$\text{angle}(\vec{a}, \vec{b}) = \text{atan2}(\text{cross2D}(\vec{a}, \vec{b}), \text{dot2D}(\vec{a}, \vec{b}))$$

Rotation of a vector by θ :

$$R_\theta(x, y) = (x \cos \theta - y \sin \theta, x \sin \theta + y \cos \theta)$$

Rotation about point $P = (p_x, p_y)$:

$$A' = P + R_\theta(A - P)$$

Lines and Intersections

Equation of a line through points A, B :

$$\vec{r}(t) = \vec{A} + t(\vec{B} - \vec{A})$$

Intersection of lines:

$$\vec{L}_1 : \vec{r} = \vec{A} + t\vec{v}, \quad \vec{L}_2 : \vec{r} = \vec{B} + s\vec{w}$$

Then

$$t = \frac{\text{cross2D}(B - A, \vec{w})}{\text{cross2D}(\vec{v}, \vec{w})}$$

Distances

Distance from point P to line AB :

$$d = \frac{|\text{cross2D}(\vec{AP}, \vec{AB})|}{|\vec{AB}|}$$

Projections and Reflections (2D)

Projection of $\vec{a}$ onto unit direction $\hat{b}$:

$$\vec{a}_{\parallel} = (\vec{a} \cdot \hat{b}) \hat{b}$$

Perpendicular component:

$$\vec{a}_{\perp} = \vec{a} - \vec{a}_{\parallel}$$

Reflection of $\vec{a}$ about unit normal $\hat{n}$:

$$\vec{a}' = \vec{a} - 2(\vec{a} \cdot \hat{n}) \hat{n}$$

Projection of $\vec{a}$ onto line with normal $\hat{n}$:

$$\vec{a}' = \vec{a} - (\vec{a} \cdot \hat{n}) \hat{n}$$

12 Z - Templates

12.1 stress

```
#include <bits/stdc++.h>
using namespace std;
const string NAME = "template";
const int NTEST = 100;

static mt19937_64
    rng(chrono::steady_clock::now().time_since_epoch().count());
#define rando(1, r) uniform_int_distribution<int>(1,
    r)(rng)

void sinh(){
    ofstream fout("inp.txt");
    fout << rando(1, 8) << '\n';
    fout.close();
}

int main()
{
    srand(time(NULL));
    for (int iTest = 1; iTest <= NTEST; iTest++)
    {
        sinh();

        // Change to "./" for Linux
        system((NAME + ".exe").c_str());
        system((NAME + "_trau.exe").c_str());

        // Use diff instead of fc for Linux
        if (system(("fc " + NAME + ".out " + NAME +
            ".ans").c_str()) != 0)
        {
            cout << "Test " << iTest << ": WRONG!\n";
            return 0;
        }
        cout << "Test " << iTest << ": CORRECT!\n";
    }
    cout << "CubU\n";
    return 0;
}
```
